

Laboratório de desenvolvimento de software

Java Swing

Java Swing

- O Java Swing é uma biblioteca gráfica fornecida pela linguagem de programação Java para criar interfaces gráficas de usuário (GUIs).
- Ele permite criar aplicativos com uma aparência e comportamento visual atraentes e interativos.
- O Swing é parte da Java Foundation Classes (JFC) e oferece uma ampla gama de componentes GUI, como botões, caixas de texto, painéis e janelas, que podem ser usados para construir interfaces ricas e dinâmicas.

Java Swing

- Vantagens
 - Plataforma Independente: As aplicações Swing funcionam em várias plataformas sem a necessidade de reescrever o código.
 - Componentes Personalizáveis: É possível personalizar a aparência e o comportamento dos componentes Swing para atender às necessidades específicas do projeto.
 - Rica Variedade de Componentes: Swing oferece uma vasta gama de componentes que podem ser combinados para criar interfaces complexas.
 - Interface Responsiva: As aplicações Swing são conhecidas por serem responsivas e rápidas.
 - Look and Feel Consistente: Swing permite aplicar diferentes "Look and Feels" para adaptar a aparência da aplicação ao sistema operacional.

Java Swing

- Desvantagens
 - Aparência e Experiência de Usuário: O Java Swing não oferece uma aparência nativa em todas as plataformas. Isso significa que, dependendo do sistema operacional, as interfaces criadas com Swing podem parecer deslocadas e diferentes das interfaces nativas, o que pode afetar a experiência do usuário.
 - Desempenho: O Java Swing pode ser mais pesado em termos de recursos e desempenho em comparação com tecnologias de GUI mais modernas. Isso pode levar a uma menor responsividade e atrasos perceptíveis em interfaces mais complexas.
 - Falta de Recursos Modernos: O Swing carece de recursos modernos de design e interação presentes em outras bibliotecas e frameworks mais recentes. Isso pode limitar a capacidade de criar interfaces altamente interativas e visualmente atraentes.
- Entre outras...

Java Swing

- Porém, o Java Swing é um método que facilmente poderemos compreender o desenvolvimento de software
- Utilizar uma conexão com banco de dados para a integração e o desenvolvimento de software.
- Utilizar componentes e interação com o usuário.
- Entre outros pontos...

Java Swing

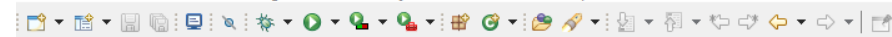
- Temos a possibilidade de utilizar o Eclipse e o Netbeans
 - O eclipse necessita da instalação do WindowBuilder
 - WindowBuilder é uma ferramenta que possibilita a construção de telas empregando as principais APIs gráficas do Java Desktop.
- Já no Netbeans, o Swing já vem instalado na versão tradicional
 - Maior facilidade e organização da interface e código fonte

Java Swing

- Para a introdução e visão geral, iremos iniciar utilizando o Eclipse e efetuar a instalação dos pacotes necessários...

Java Swing

- Para instalar o WindowBuilder no eclipse:
 - Clique em “Help”
 - Após clique em “EclipseMarketplace”
 - Pesquise por “WindowBuilder”



Package Explorer



There are no projects in your workspace.
To add a project:

[Create a Java project](#)[Create a project...](#)[Import projects...](#)

 Eclipse Marketplace

Eclipse Marketplace
Select solutions to install. Press Install Now to proceed with installation.
Press the "more info" link to learn more about a solution.

Search

Recent

Popular

Favorites

Installed

Giving IoT an Edge

Find:

**WindowBuilder Current**
WindowBuilder is composed of SWT Designer and Swing Designer and makes it very easy to create Java GUI applications without spending a lot of time writing code.... [more info](#)
by Eclipse Foundation, EPL
[SWT swing wysiwyg graphical WindowBuilder](#)
★ 967

**WindowBuilder Nightly Build Nightly**
Nightly Build! Install if you want to use the latest patches before a release is available. It can be unstable. WindowBuilder is composed of SWT Designer and... [more info](#)
by Eclipse Foundation, EPL
[SWT swing wysiwyg graphical WindowBuilder](#)
★ 49

**Spark Builder Generator 0.0.28**
Generates a builder according to the GoF pattern for Java domain objects. Features Generates a builder with custom name patterns Can generate staged builder ... [more info](#)
by Helo Spark, MIT
[Builder Pattern builder code generator SparkTools design pattern](#)
★ 110

Marketplaces


Outline



There is no active editor that provides an outline.

Problems Javadoc Declaration Console

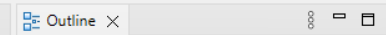
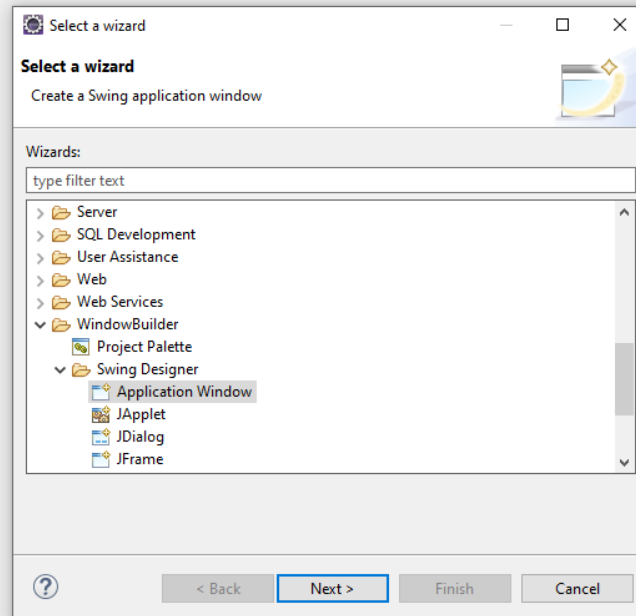
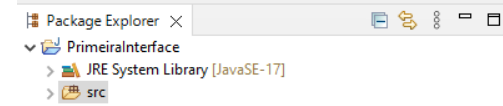
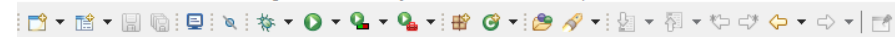
No consoles to display at this time.

Java Swing

- Criando um projeto com WindowBuilder
 - Primeiro, crie um projeto
 - File -> New -> Java Project
 - Defina o nome do projeto, como por exemplo “PrimeiraInterface” e clique em Finish
 - Neste projeto, ainda não temos nada, nem uma classe, nem uma main, nem nada...

Java Swing

- Para isso, precisamos:
 - Clicar com o botão direito em cima de “src”
 - Clicar em “New”
 - Clicar em “Other”
 - Agora, precisamos pesquisar por “WindowBuilder” -> “Swing Designer”
 - Selecionar “Application Window” e clicar “Next”
 - Definimos o nome do “Package” e definimos o nome, como por exemplo, “PrimeiraInterface”



There is no active editor that provides an outline.



Package Explorer

PrimeiralInterface

> JRE System Library [JavaSE-17]

> src

Outline

There is no active editor that provides an outline.

New Swing Application

Create Application
Create a Swing application window.

Source folder: PrimeiralInterface/src

Package: pkg

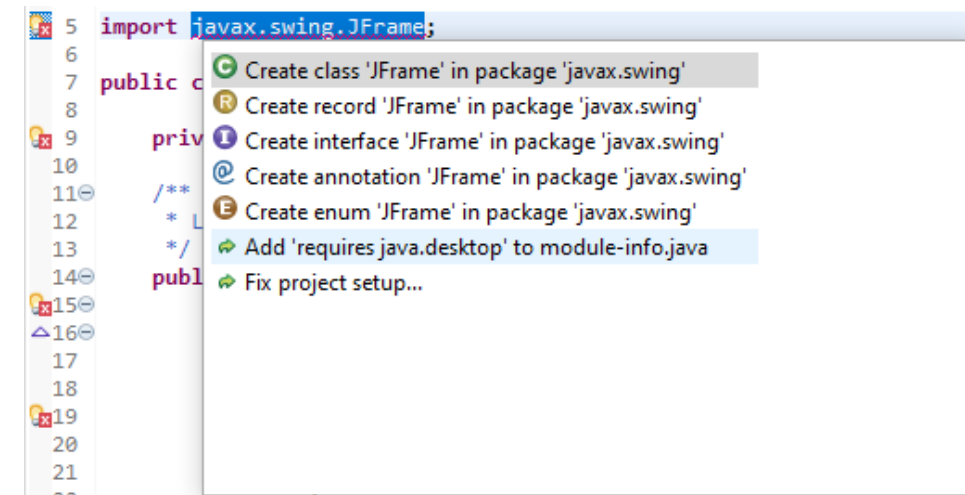
Name: PrimeiralInterface

Problems @ Javadoc Declaration Console

No consoles to display at this time.

Java Swing

- Possivelmente, pode ter ocorrido um erro...
- Este erro ocorre pois o eclipse por algum motivo não adicionou a WindowBuilder dentro do “module-info.java” e acaba não encontrando os imports das bibliotecas.
- Para corrigir, basta clicar no erro
- Clicar em “Add ‘requires java.desktop...’ “



eclipse-workspace - PrimeiralInterface/src/pkg/PrimeiralInterface.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- PrimeiralInterface
 - JRE System Library [JavaSE-17]
 - src
 - pkg
 - PrimeiralInterface.java
 - module-info.java

PrimeiralInterface.java

```
1 package pkg;
2
3 import java.awt.EventQueue;
4
5 import javax.swing.JFrame;
6
7 public class PrimeiralInterface {
8
9     private void initialize() {
10
11         /**
12          * Create the application.
13          */
14         public PrimeiralInterface() {
15             initialize();
16         }
17
18         /**
19          * Initialize the contents of the frame.
20          */
21         private void initialize() {
22             frame = new JFrame();
23             frame.setBounds(100, 100, 450, 300);
24             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25         }
26     }
27 }
28
29 /**
30  * Create the application.
31  */
32 public PrimeiralInterface() {
33     initialize();
34 }
35
36 /**
37  * Initialize the contents of the frame.
38  */
39 private void initialize() {
40     frame = new JFrame();
41     frame.setBounds(100, 100, 450, 300);
42     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43 }
44 }
```

Outline

- pkg
 - PrimeiralInterface
 - frame : JFrame
 - main(String[]) : void
 - new Runnable() {...}
 - PrimeiralInterface()
 - initialize() : void

Opens the new class wizard to create the type.

Package: javax.swing
public class JFrame {
}

Press 'Tab' from proposal table or click for focus

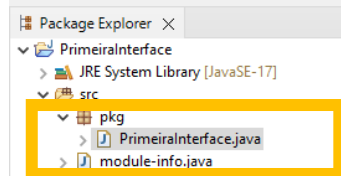
Source Design

Problems Javadoc Declaration Console

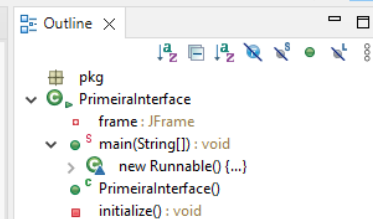
No consoles to display at this time.

The type javax.swing.JFrame is not accessible

Writable Smart Insert 5 : 26 [18]



```
1 package pkg;
2
3 import java.awt.EventQueue;
4
5 import javax.swing.JFrame;
6
7 public class PrimeiraInterface {
8
9     private JFrame frame;
10
11     /**
12      * Launch the application.
13      */
14     public static void main(String[] args) {
15         EventQueue.invokeLater(new Runnable() {
16             public void run() {
17                 try {
18                     PrimeiraInterface window = new PrimeiraInterface();
19                     window.frame.setVisible(true);
20                 } catch (Exception e) {
21                     e.printStackTrace();
22                 }
23             }
24         });
25     }
26
27     /**
28      * Create the application.
29      */
30     public PrimeiraInterface() {
31         initialize();
32     }
33
34     /**
35      * Initialize the contents of the frame.
36      */
37     private void initialize() {
38         frame = new JFrame();
39         frame.setBounds(100, 100, 450, 300);
40         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41     }
42
43 }
44
```



Console

No consoles to display at this time.

The screenshot displays the Eclipse IDE interface for the `PrimeiralInterface.java` file. The **Package Explorer** on the left shows the project structure: `PrimeiralInterface` package containing `src`, `pkg`, and `module-info.java`. The **Structure** view in the center shows the class hierarchy: `Components` (frame) containing `getContentPane()`. The **Palette** on the right lists various Java Swing components and layouts, including `System`, `Selection`, `Marquee`, `Choose component`, `Tab Order`, `Containers` (JPanel, JScrollPane, JSplitPane, JTabbedPane, JToolBar, JLayeredPane, JDesktopPane, JInternalFrame), `Layouts` (Absolute layout, FlowLayout, BorderLayout, GridLayout, GridBagLayout, CardLayout, BoxLayout, SpringLayout, FormLayout, MigLayout, GroupLayout), `Struts & Springs`, and `Components` (JLabel, JTextField, JComboBox, JButton, JCheckBox, JRadioButton, JToggleButton, JTextArea, JFormattedTextField, JPasswordField, JTextPane, JEditorPane). The **Outline** view on the far right shows the code structure: `pkg` (PrimeiralInterface) containing `frame: JFrame`, `main(String[]): void`, `new Runnable() {...}`, `PrimeiralInterface()`, and `initialize(): void`. The **Design** view in the center shows a visual representation of the GUI, which is currently empty. The **Properties** view at the bottom left is empty, displaying `<No Properties>`. The **Source** and **Declaration** tabs are visible at the bottom. The status bar at the bottom indicates `pkg.PrimeiralInterface.java - PrimeiralInterface/src`.

Java Swing

- Existem diversos componentes para serem utilizados numa interface gráfica utilizando Swing
 - JPanel: um componente que serve como um contêiner leve e flexível para agrupar outros componentes. Ele não possui uma aparência própria, mas é utilizado principalmente para organizar e agrupar outros componentes, como botões, rótulos, campos de texto, entre outros, em uma área comum dentro de uma interface gráfica.
 - JButton: Um botão clicável que executa uma ação quando pressionado.
 - JLabel: Um componente que exibe um texto ou uma imagem.
 - JTextField: Uma caixa de texto onde os usuários podem inserir texto.

Java Swing

- JCheckBox: Uma caixa de seleção que permite aos usuários escolher entre duas opções (marcada ou desmarcada).
- JRadioButton: Botões de opção que permitem aos usuários escolher uma única opção de um grupo.
- JTextArea: Componente que fornece uma área de texto multilinha, permitindo que os usuários insiram, visualizem e editem texto em um formato que vai além das limitações de um único campo de texto, como o JTextField.
- Jtable: Componente que oferece a capacidade de exibir e editar dados em forma de tabela. Ela é frequentemente utilizada para exibir informações tabulares, como listas, tabelas de dados, planilhas e outras estruturas similares.

Java Swing

- O padrão de nomenclatura de componentes no Java Swing segue convenções que ajudam a tornar o código mais legível, compreensível e organizado.
- Essas convenções facilitam a identificação dos tipos de componentes e sua finalidade dentro da interface gráfica.
- A convenção geral para nomear componentes no Java Swing é a seguinte:
 - O nome do componente começa com um prefixo que indica o tipo de componente:
 - btn para botões (JButton)
 - lbl para rótulos (JLabel)

Java Swing

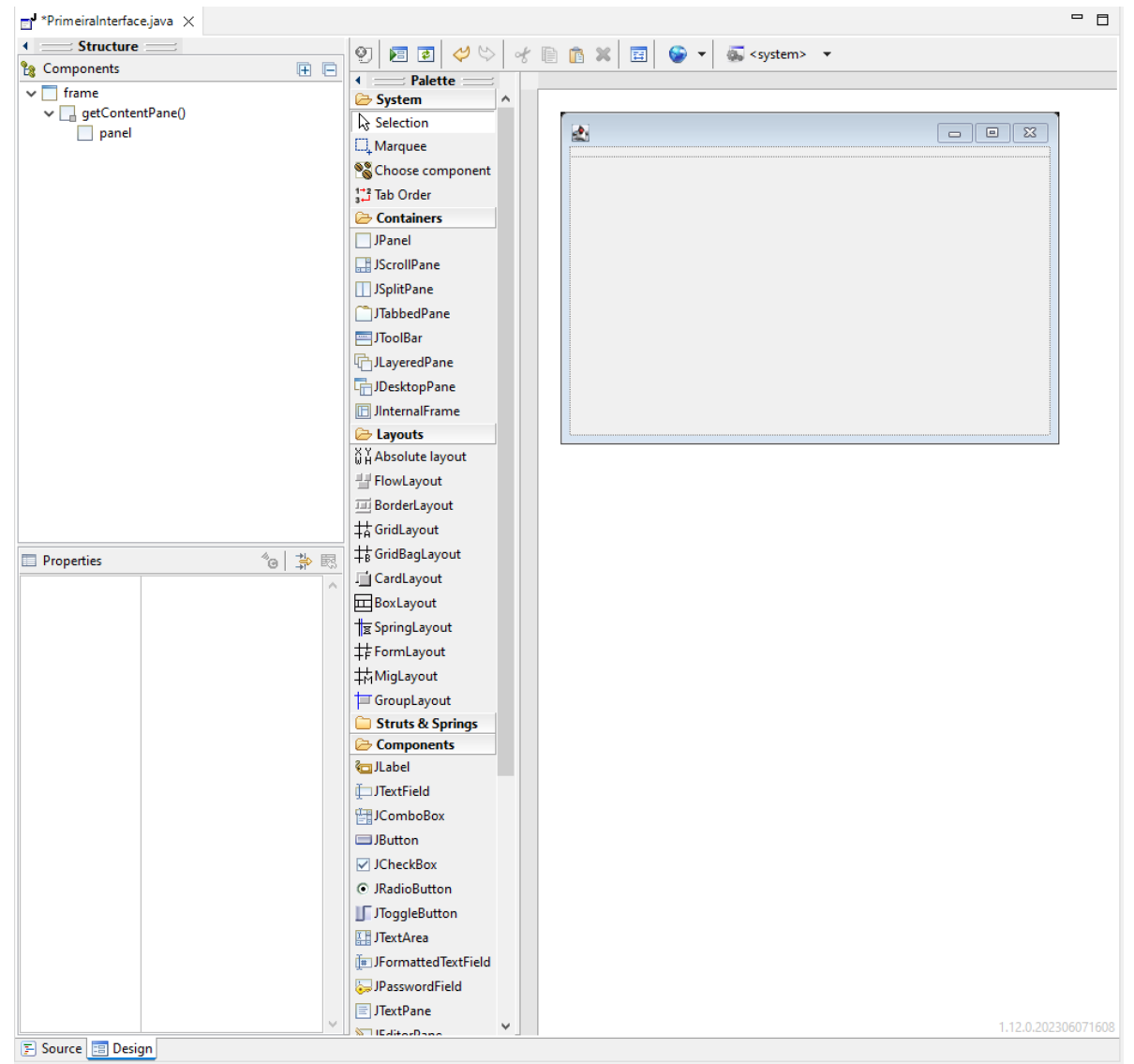
- O nome do componente começa com um prefixo que indica o tipo de componente:
 - txt para campos de texto (JTextField)
 - cmb para caixas de seleção (JComboBox)
 - chk para caixas de seleção (JCheckBox)
 - rad para botões de opção (JRadioButton)
 - tbl para tabelas (JTable)
 - pnl para painéis (JPanel)

Java Swing

- Após o prefixo, você pode usar um nome descritivo para indicar a função ou finalidade do componente.
- Por exemplo: `btnSalvar`, `lblMensagem`, `txtNome`, `cmbCidades`.

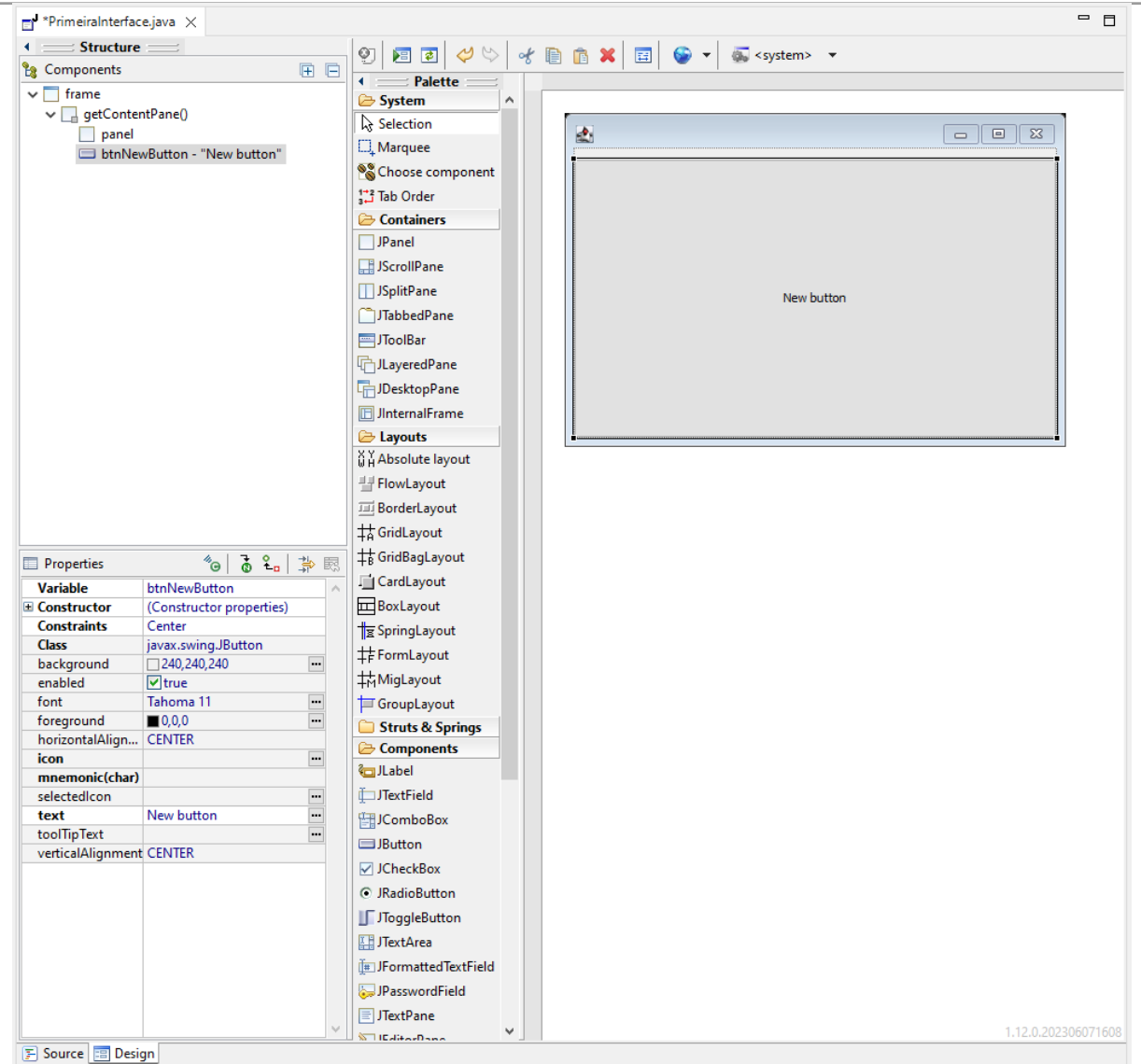
Java Swing

- Adicione o JPanel



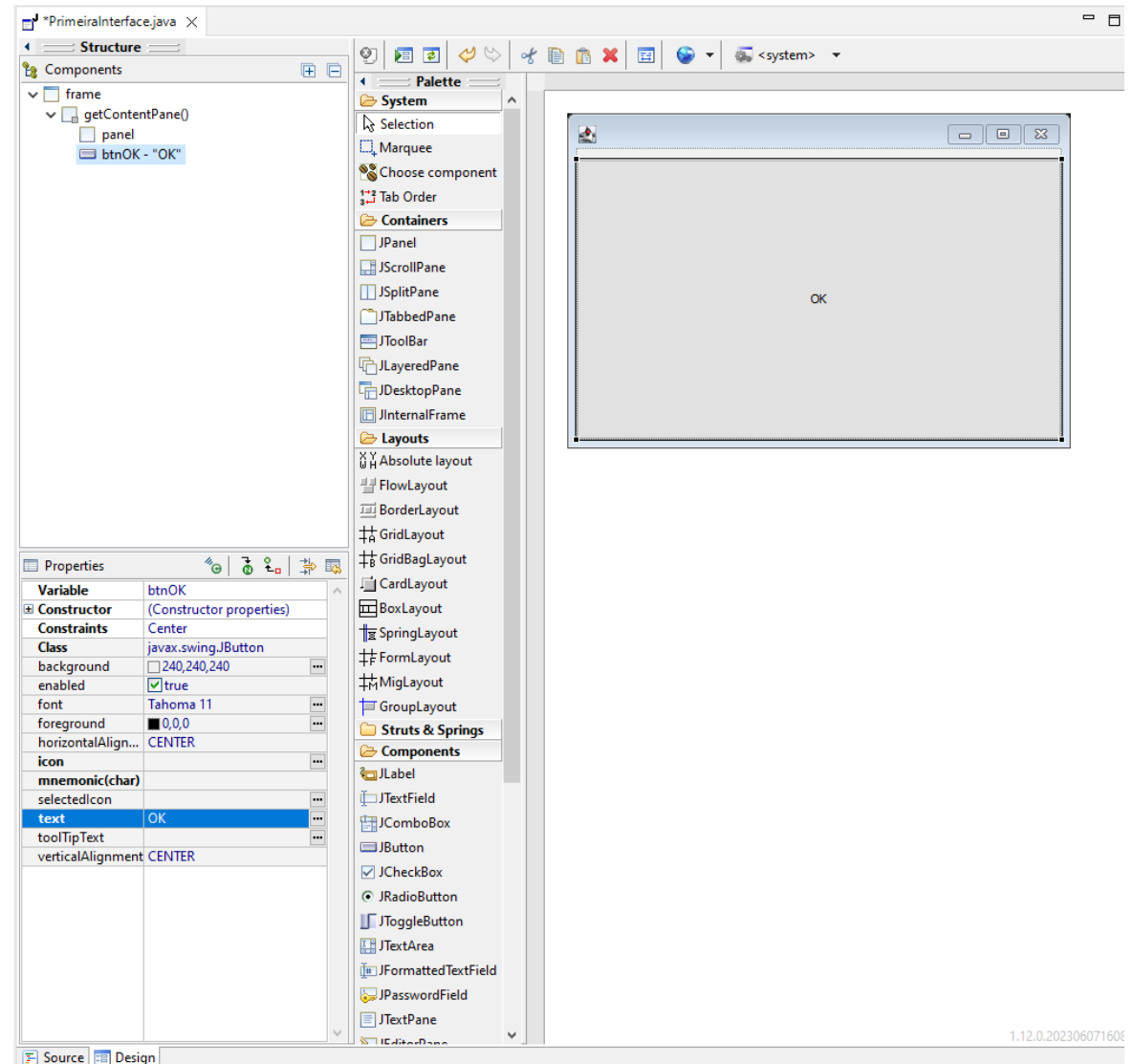
Java Swing

- Adicione o JPanel
- Após, adicione o JButton



Java Swing

- Adicione o JPanel
- Após, adicione o JButton
- Conforme o padrão, defina o nome do botão



Java Swing

- Adicione o JPanel
- Após, adicione o JButton
- Conforme o padrão, defina o nome do botão
- Dê dois cliques no botão e:

```
PrimeiraInterface.java X
1 package pkg;
2
3 import java.awt.EventQueue;
4
5 import javax.swing.JFrame;
6 import javax.swing.JButton;
7 import java.awt.BorderLayout;
8 import javax.swing.JPanel;
9 import java.awt.event.ActionListener;
10 import java.awt.event.ActionEvent;
11
12 public class PrimeiraInterface {
13
14     private JFrame frame;
15
16     /**
17      * Launch the application.
18      */
19     public static void main(String[] args) {
20         EventQueue.invokeLater(new Runnable() {
21             public void run() {
22                 try {
23                     PrimeiraInterface window = new PrimeiraInterface();
24                     window.frame.setVisible(true);
25                 } catch (Exception e) {
26                     e.printStackTrace();
27                 }
28             }
29         });
30     }
31
32     /**
33      * Create the application.
34      */
35     public PrimeiraInterface() {
36         initialize();
37     }
38
39     /**
40      * Initialize the contents of the frame.
41      */
42     private void initialize() {
43         frame = new JFrame();
44         frame.setBounds(100, 100, 450, 300);
45         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46
47         JPanel panel = new JPanel();
48         frame.getContentPane().add(panel, BorderLayout.NORTH);
49
50         JButton btnOK = new JButton("OK");
51         btnOK.addActionListener(new ActionListener() {
52             public void actionPerformed(ActionEvent e) {
53             }
54         });
55         frame.getContentPane().add(btnOK, BorderLayout.CENTER);
56     }
57 }
58
59
```

Java Swing

- Ele abrira novamente o código e automaticamente ele irá criar um método chamado “actionPerformed”
- Este método será executado toda vez que o botão for pressionado...

```
43- private void initialize() {  
44-     frame = new JFrame();  
45-     frame.setBounds(100, 100, 450, 300);  
46-     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
47-  
48-     JPanel panel = new JPanel();  
49-     frame.getContentPane().add(panel, BorderLayout.NORTH);  
50-  
51-     JButton btnOK = new JButton("OK");  
52-     btnOK.addActionListener(new ActionListener() {  
53-         public void actionPerformed(ActionEvent e) {  
54-             JOptionPane.showMessageDialog(frame, "Oi mundo!");  
55-         }  
56-     });  
57-     frame.getContentPane().add(btnOK, BorderLayout.CENTER);  
58- }  
59-  
60-  
61- }
```

Java Swing

- Tentem redimensionar o botão...
 - O que acontece?

Java Swing

- Tentem redimensionar o botão...
 - O que acontece?
 - Ele fica preso, não altera a posição...
 - Pra isso, precisamos abordar sobre os layouts possíveis

Layouts

- Existem vários layouts disponíveis no Java Swing, cada um com características e comportamentos específicos. Aqui estão alguns dos layouts mais comuns:
- **FlowLayout:**
 - Posiciona os componentes em uma única linha, em ordem de inserção.
 - Se os componentes não couberem em uma linha, eles são distribuídos para a próxima linha.
 - Adequado para caixas de diálogo simples ou barras de ferramentas.

Layouts

- **BorderLayout:**
 - Divide o contêiner em cinco regiões: norte, sul, leste, oeste e centro.
 - Cada região pode conter apenas um componente.
 - É útil para criar layouts mais complexos, onde os componentes podem se expandir ou contrair conforme o espaço disponível.

Layouts

- **GridLayout:**
 - Organiza os componentes em uma grade retangular de células.
 - Cada célula pode conter um único componente.
 - Adequado para criar interfaces em formato de tabela, como calculadoras ou jogos de tabuleiro.

Layouts

- **BoxLayout:**
 - Posiciona os componentes em uma linha (horizontal ou vertical).
 - Permite o alinhamento dos componentes no início, centro ou final da linha.
 - Útil quando você precisa de um controle mais preciso sobre o alinhamento dos componentes.

Layouts

- **GridBagLayout:**
 - Oferece um alto nível de flexibilidade e controle sobre a posição e o tamanho dos componentes.
 - Pode ser mais complexo de usar devido às muitas opções de configuração.
 - Útil para layouts personalizados onde o posicionamento exato dos componentes é crucial.

Layouts

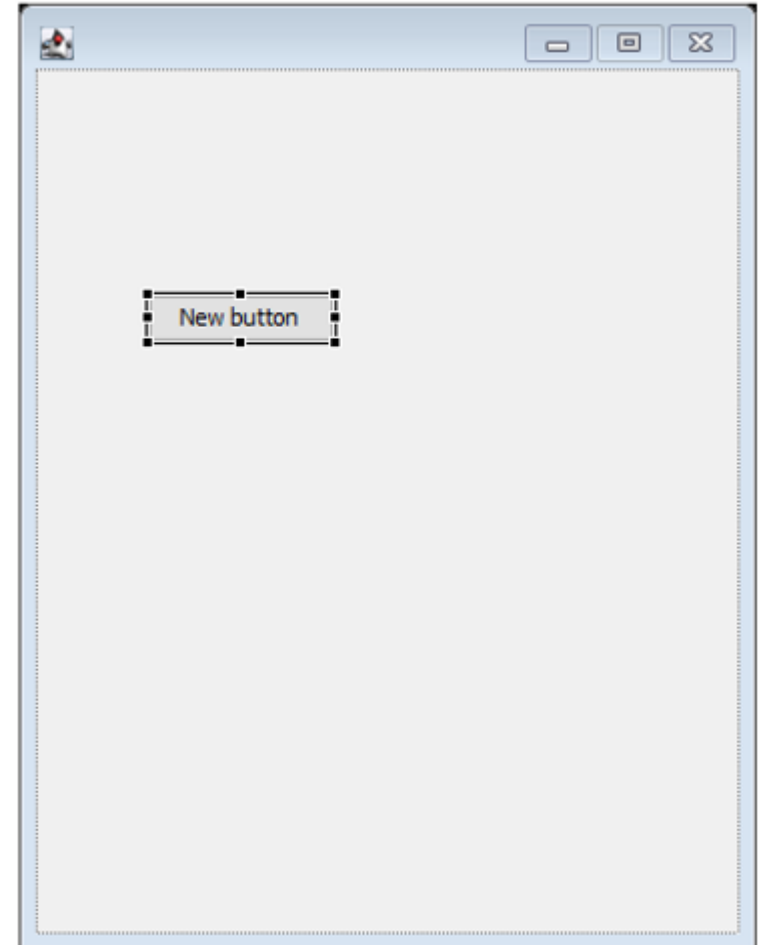
- **CardLayout:**
 - Permite alternar entre diferentes painéis, onde apenas um é visível por vez.
 - É adequado para a criação de painéis de navegação ou etapas em aplicativos com várias telas.

Layouts

- Adicione “Absolute layout”
 - E adicione outro “Jbutton”
 - E agora?

Layouts

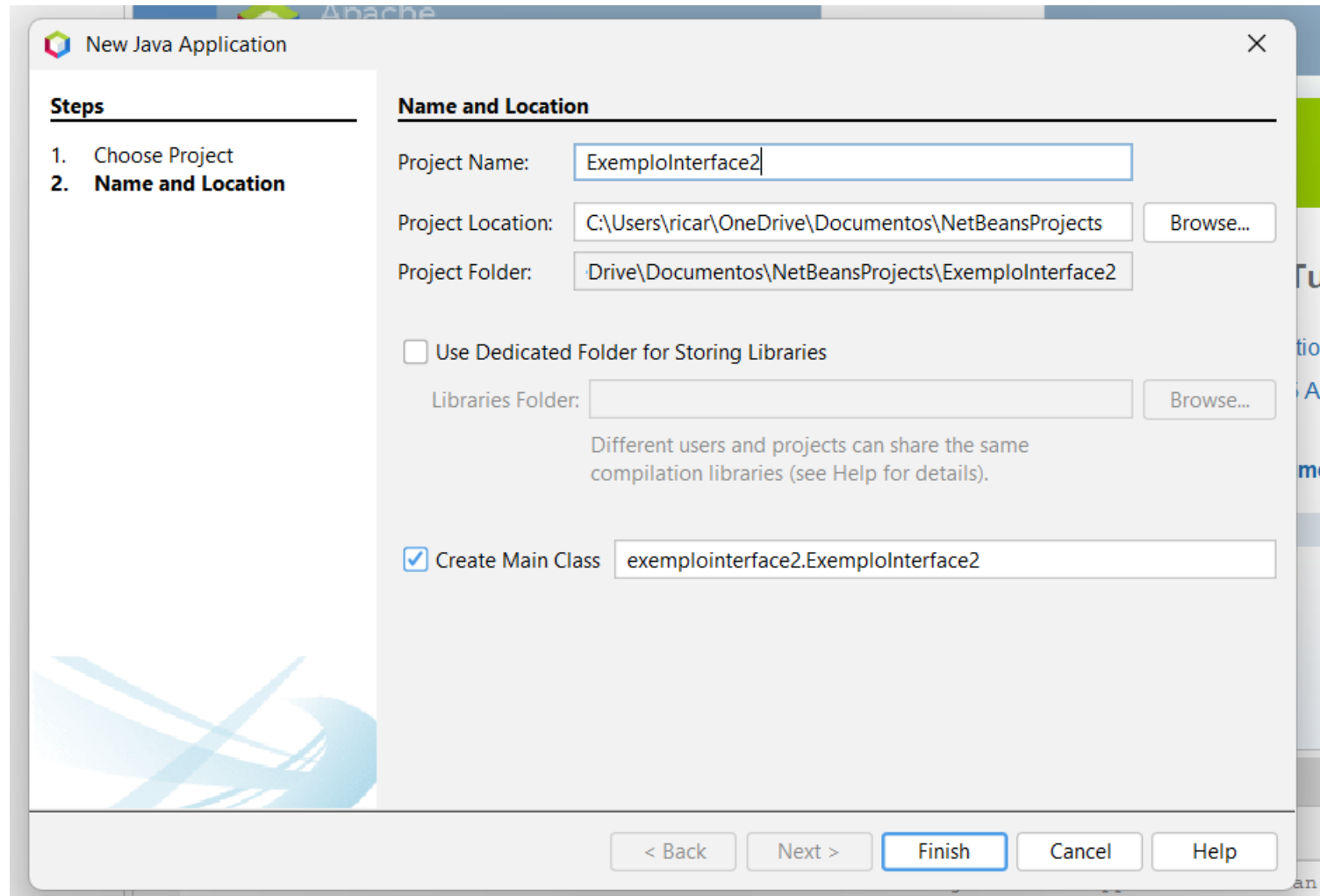
- Adicione “Absolute layout”
 - E adicione outro “Jbutton”
 - E agora?



Java Swing

- Agora, abra o NetBeans
 - Clique em “File” -> “New Project”
 - Selecione “Java with Ant” e “Java Application”
 - Defina o nome para o projeto, por exemplo, ExemploInterface2 e desmarque a opção “Create Main Class”

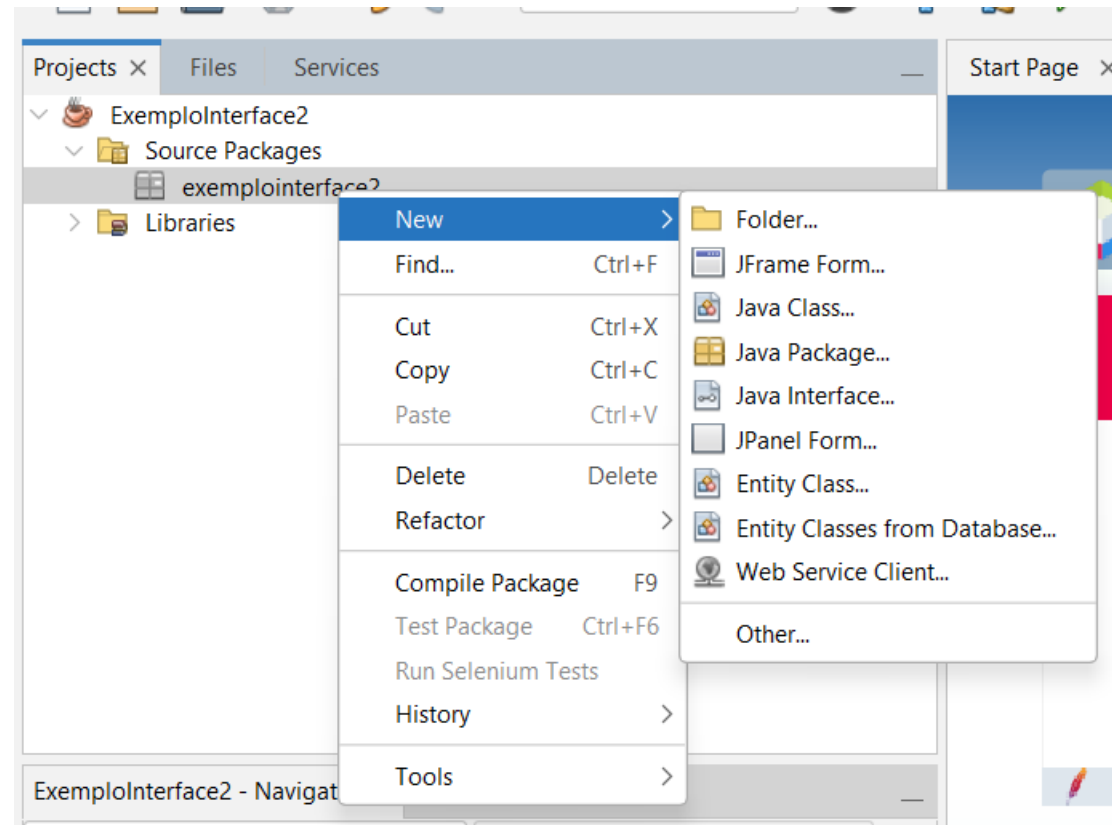
Java Swing



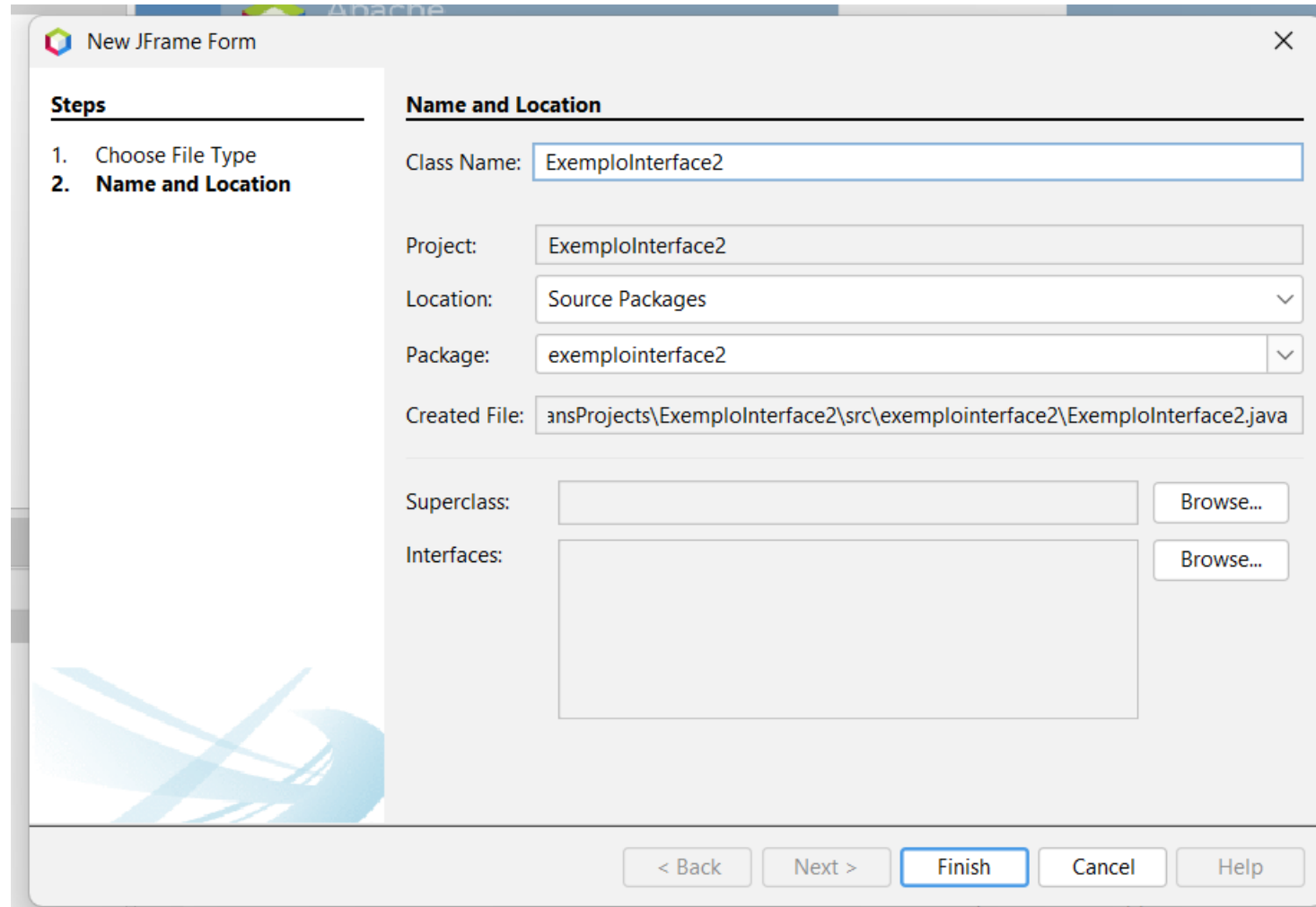
Java Swing

- Clique com botão direito no nome do pacote
 - “New” -> “Jframe Form”

Java Swing



Java Swing



The screenshot shows a 'New JFrame Form' dialog box with a 'Steps' sidebar on the left and a 'Name and Location' section on the right. The 'Steps' sidebar lists two steps: '1. Choose File Type' and '2. Name and Location', with the second step being the active one. The 'Name and Location' section contains several input fields: 'Class Name' (ExemplInterface2), 'Project' (ExemplInterface2), 'Location' (Source Packages), 'Package' (exemplinterface2), and 'Created File' (ansProjects\ExemplInterface2\src\exemplinterface2\ExemplInterface2.java). There are also 'Superclass' and 'Interfaces' sections, each with a text area and a 'Browse...' button. At the bottom, there are navigation buttons: '< Back', 'Next >', 'Finish' (highlighted), 'Cancel', and 'Help'.

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name:

Project:

Location:

Package:

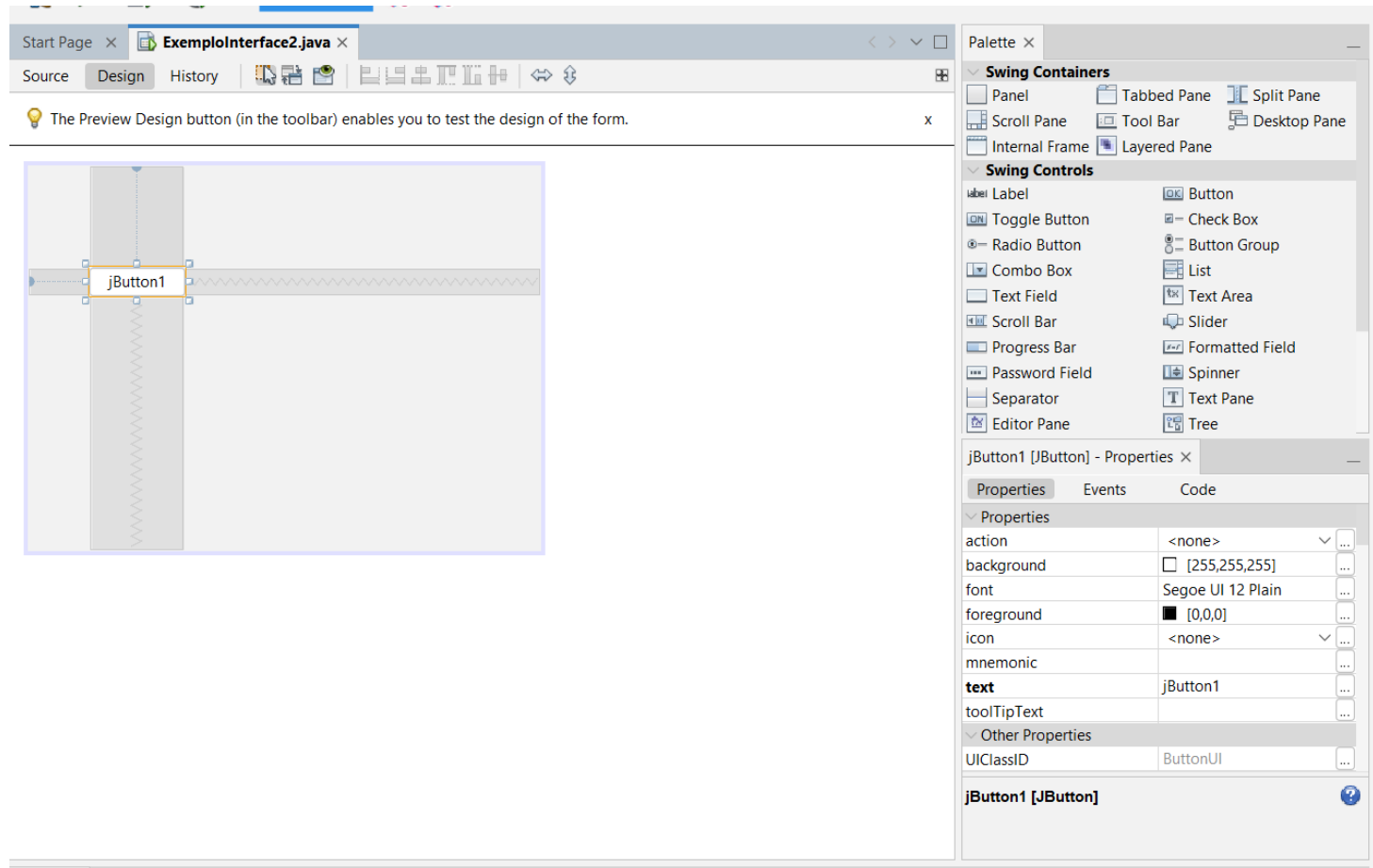
Created File:

Superclass:

Interfaces:

Java Swing

- Agora, adicione um botão no jForm

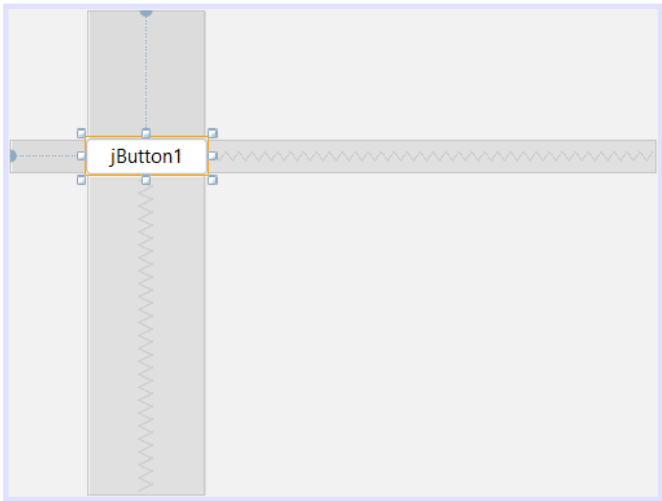


Java Swing

Start Page x **ExemplInterface2.java** x

Source Design History

The Preview Design button (in the toolbar) enables you to test the design of the form.



Palette

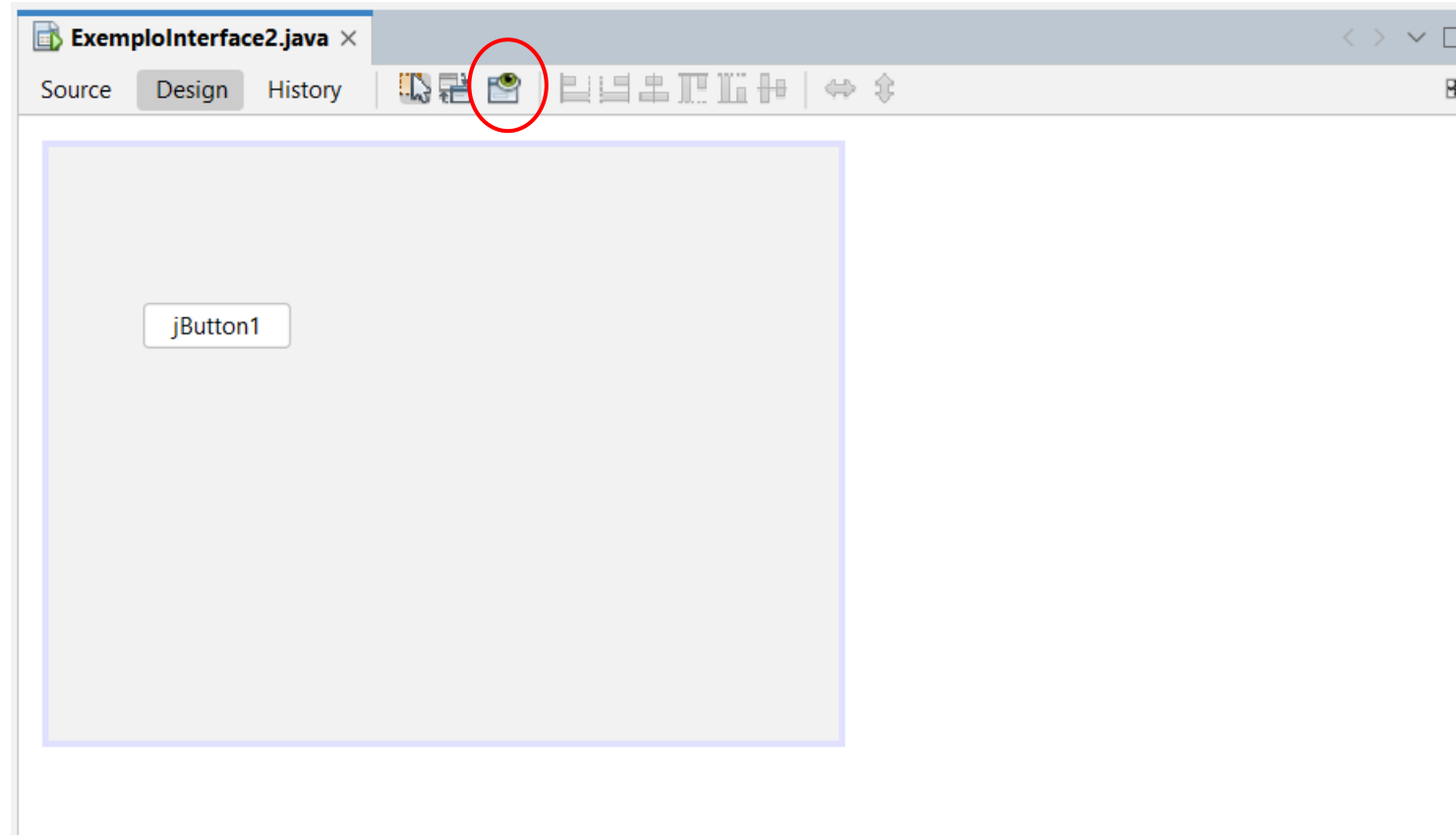
- Swing Containers**
 - Panel
 - Tabbed Pane
 - Split Pane
 - Scroll Pane
 - Tool Bar
 - Desktop Pane
 - Internal Frame
 - Layered Pane
- Swing Controls**
 - Label
 - Toggle Button
 - Radio Button
 - Combo Box
 - Text Field
 - Scroll Bar
 - Progress Bar
 - Password Field
 - Separator
 - Editor Pane
 - Button
 - Check Box
 - Button Group
 - List
 - Text Area
 - Slider
 - Formatted Field
 - Spinner
 - Text Pane
 - Tree

jButton1 [JButton] - Properties

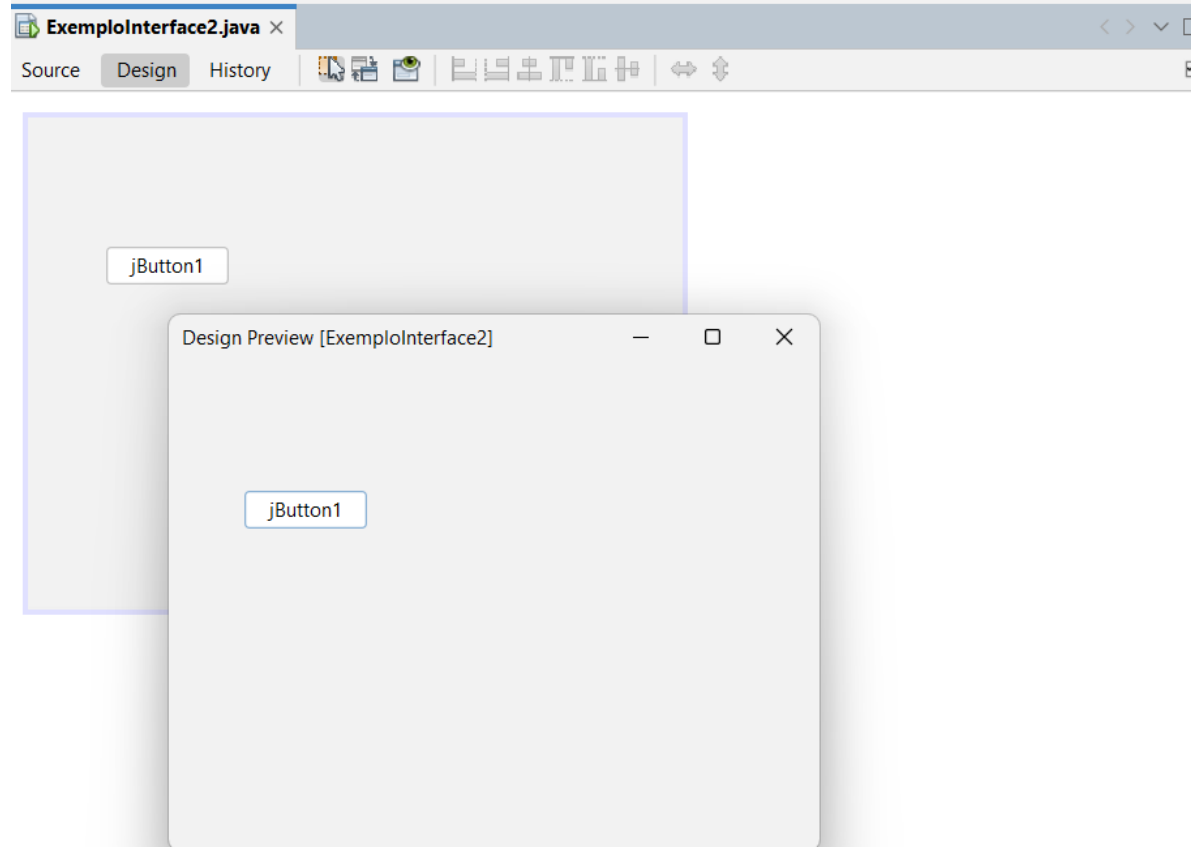
Properties	Events	Code
Properties		
action	<none>	...
background	[255,255,255]	...
font	Segoe UI 12 Plain	...
foreground	[0,0,0]	...
icon	<none>	...
mnemonic		...
text	jButton1	...
toolTipText		...
Other Properties		
UIClassID	ButtonUI	...

jButton1 [JButton]

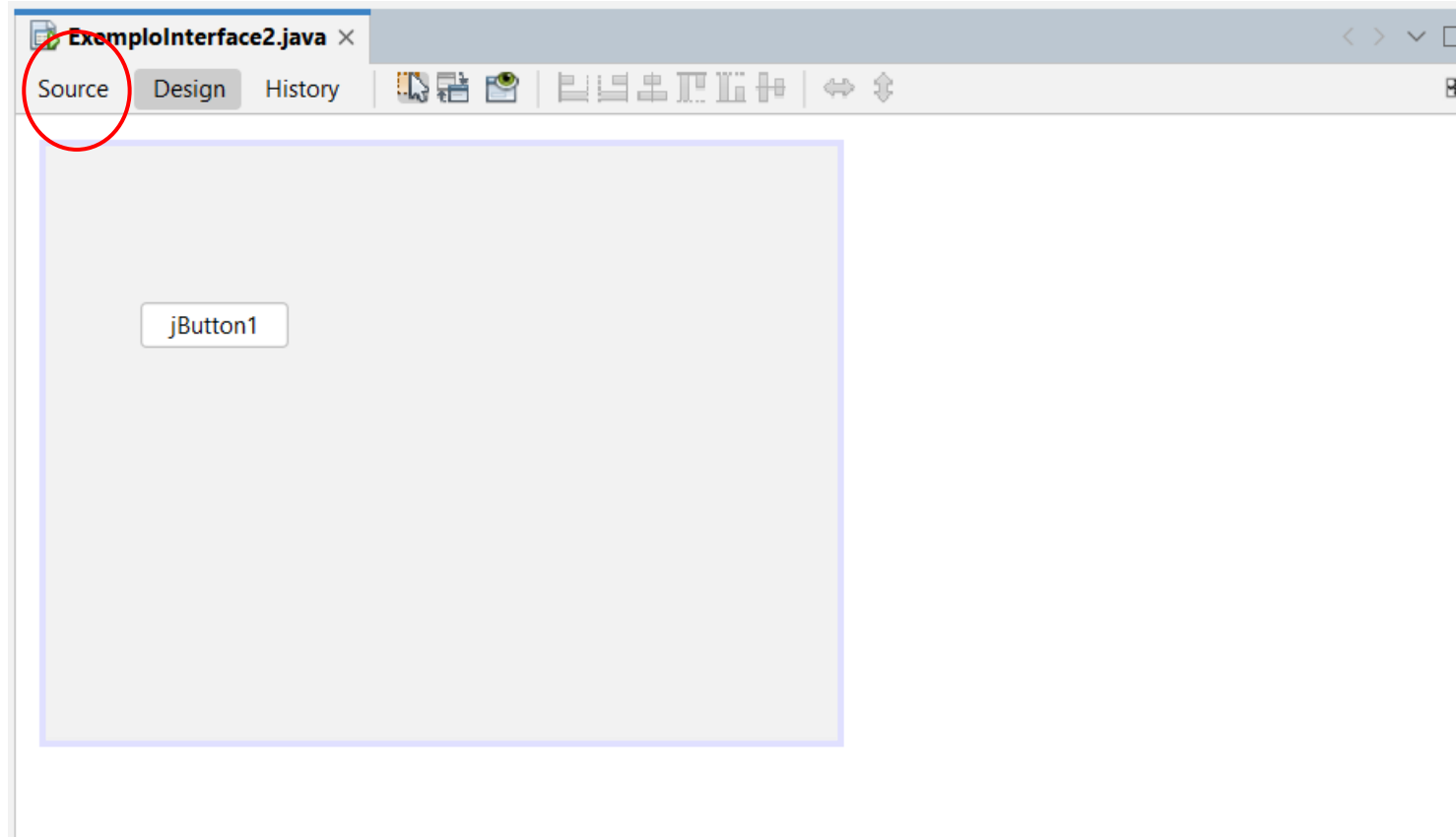
Java Swing



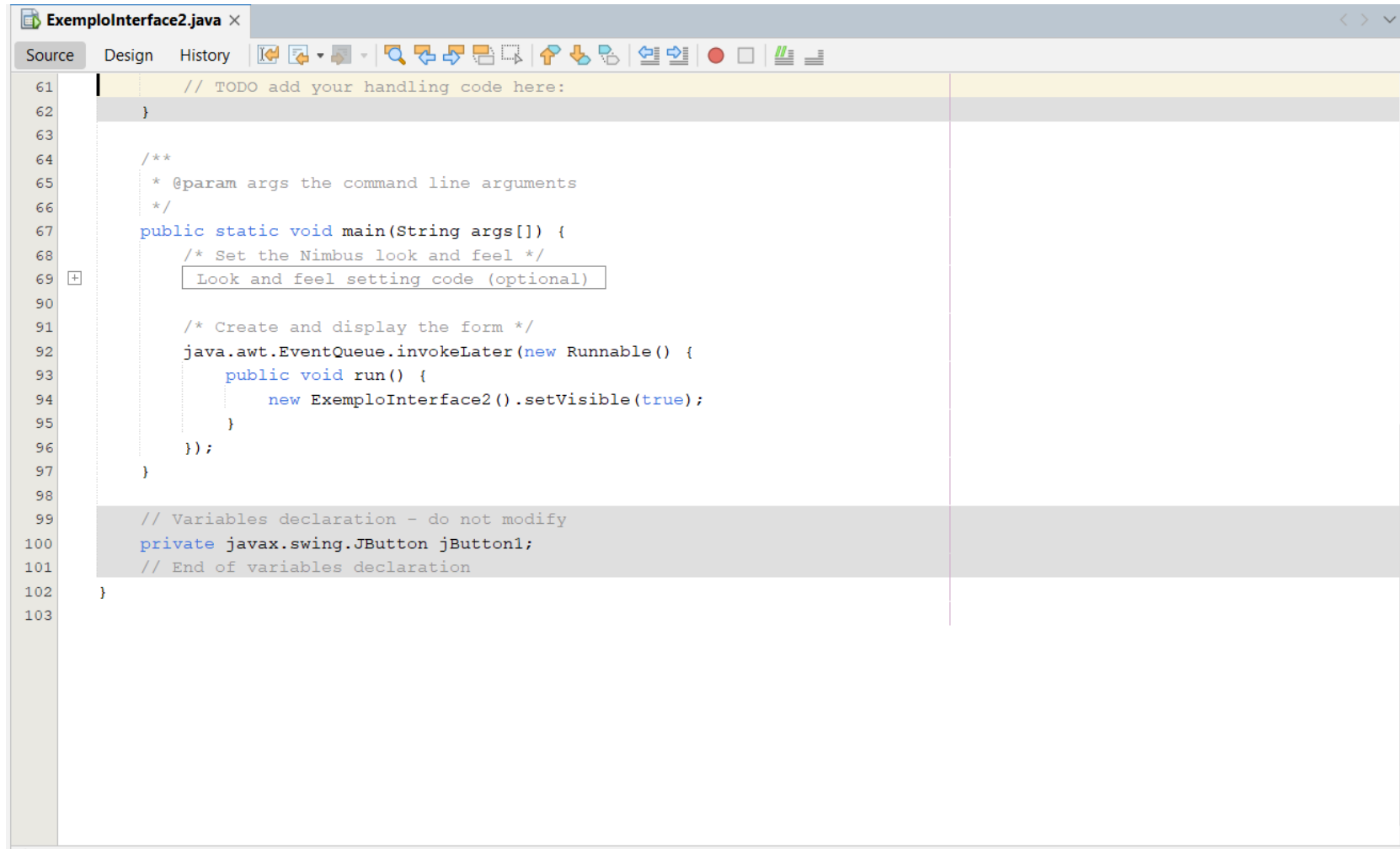
Java Swing



Java Swing



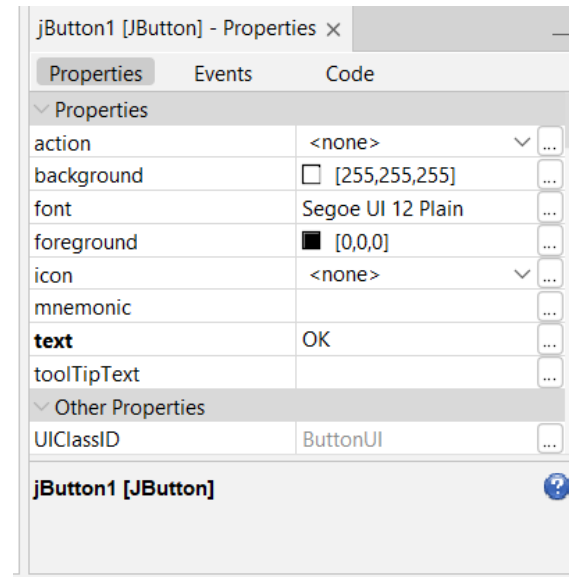
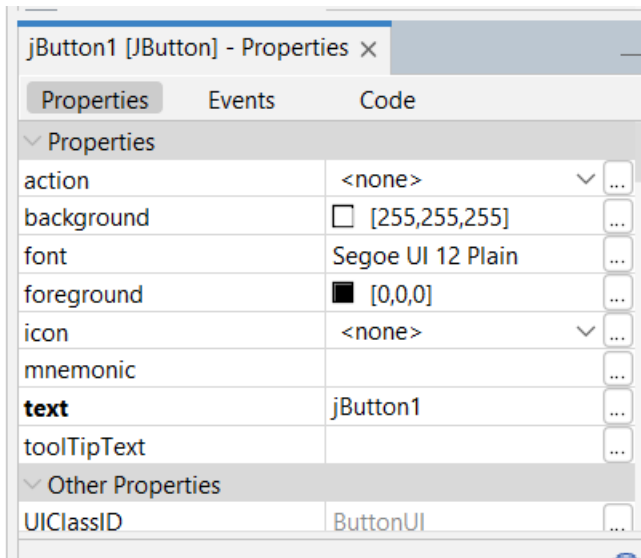
Java Swing



```
ExemploInterface2.java x
Source Design History
61 // TODO add your handling code here:
62 }
63
64 /**
65  * @param args the command line arguments
66  */
67 public static void main(String args[]) {
68     /* Set the Nimbus look and feel */
69     Look and feel setting code (optional)
70
71     /* Create and display the form */
72     java.awt.EventQueue.invokeLater(new Runnable() {
73         public void run() {
74             new ExemploInterface2().setVisible(true);
75         }
76     });
77 }
78
79 // Variables declaration - do not modify
80 private javax.swing.JButton jButton1;
81 // End of variables declaration
82 }
83
```

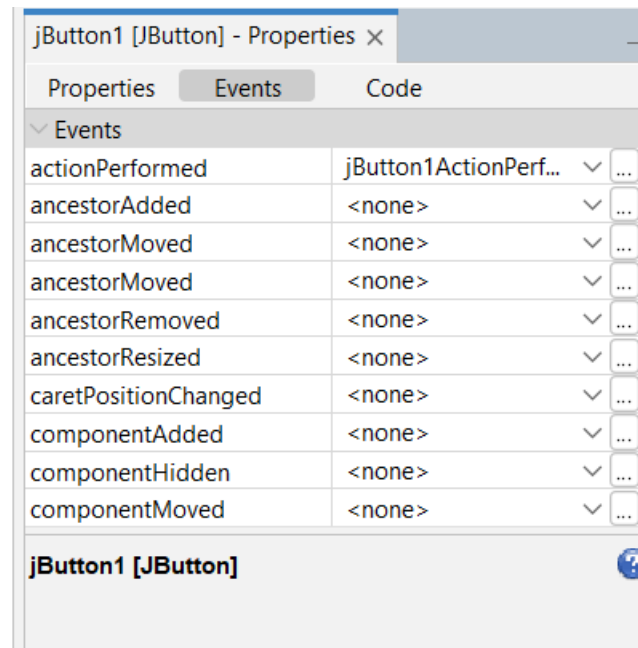

Java Swing

- Agora, procure por “text” nas propriedades do JButton e altere para OK



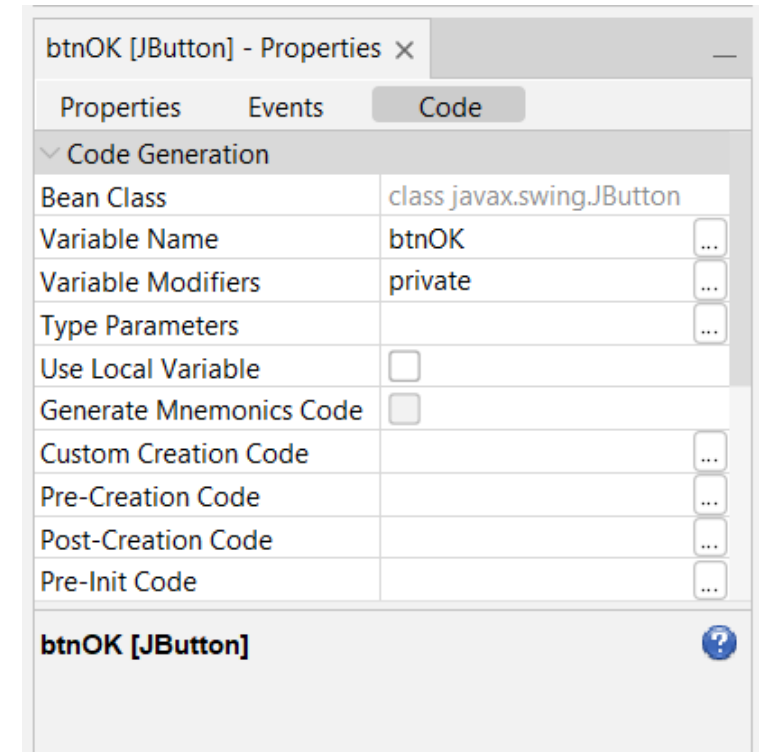
Java Swing

- Além disso, existem mais duas abas de cada componente
- A segunda é “Events”, onde é possível programar qualquer tipo de evento para cada componente



Java Swing

- Além disso, existem mais duas abas de cada componente
- O terceiro é “Code”, onde é possível verificar e alterar o nome do componente e seguir a convenção
 - Altere o nome para “btnOK”
- Nesta aba também é possível alterar a visibilidade do botão, parâmetros nas chamadas de métodos entre outras coisas...



Java Swing

- Agora, dê dois cliques no botão e adicione o seguinte código:

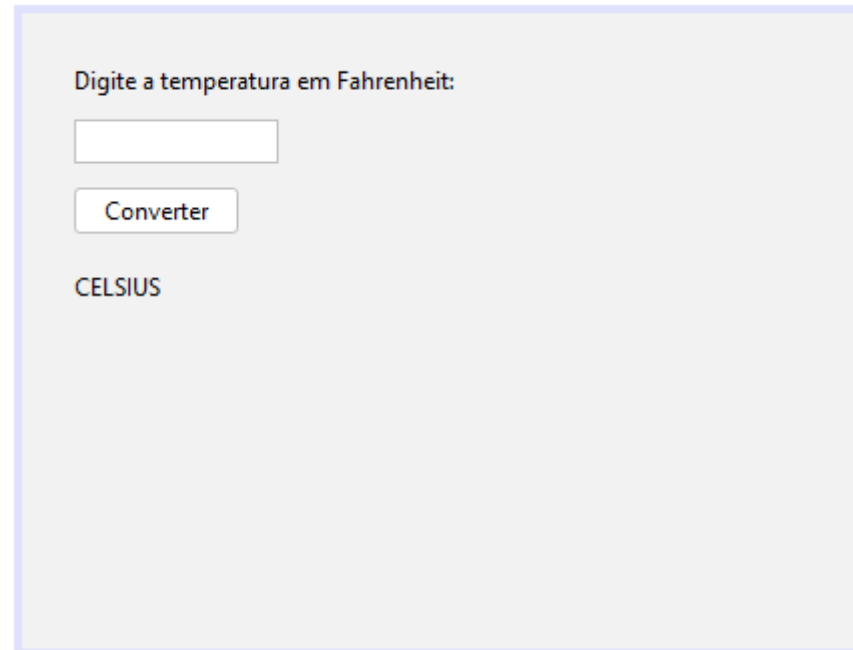
```
13 public class ExemploInterface2 extends javax.swing.JFrame {
14
15     /**
16      * Creates new form ExemploInterface2
17      */
18     public ExemploInterface2() {
19         initComponents();
20     }
21
22     /**
23      * This method is called from within the constructor to initialize the form.
24      * WARNING: Do NOT modify this code. The content of this method is always
25      * regenerated by the Form Editor.
26      */
27     @SuppressWarnings("unchecked")
28     Generated Code
61
62     private void btnOKActionPerformed(java.awt.event.ActionEvent evt) {
63         JOptionPane.showMessageDialog(parentComponent: null, message: "Oi mundo!");
64     }
65
```

Java Swing

- Crie um formulário com um JTextField que leia uma temperatura em Fahrenheit e calcule a temperatura em Celsius após clicar no botão.
- Apresente o resultado em um JLabel.

Java Swing

- Crie um formulário com um JTextField que leia uma temperatura em Fahrenheit e calcule a temperatura em Celsius após clicar no botão.
- Apresente o resultado em um JLabel.



Digite a temperatura em Fahrenheit:

Converter

CELSIUS

Java Swing

- Crie um formulário com um JTextField que leia uma temperatura em Fahrenheit e calcule a temperatura em Celsius após clicar no botão.
- Apresente o resultado em um JLabel.

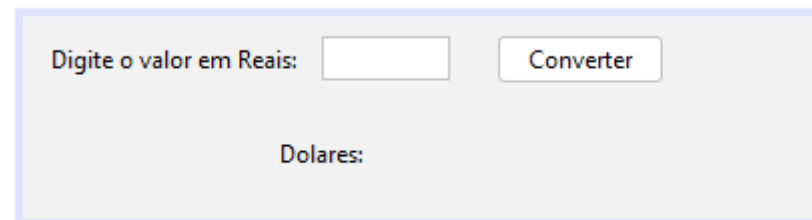
```
private void btnOKActionPerformed(java.awt.event.ActionEvent evt) {  
    double temFare, tempCelsius;  
    temFare = Double.parseDouble(txtFare.getText());  
    tempCelsius = (temFare - 32) * 5/9;  
    //lblCelsius.setText(String.valueOf(tempCelsius));  
    lblCelsius.setText("CELSIUS: "+tempCelsius);  
}
```

Atividade

- Criar um conversor de real para dólar.
- Crie um TextField que leia o valor em real e faça a conversão para dólar e apresente em um label.
- Cotação: 1 dólar = R\$5,44

Atividade

- Criar um conversor de real para dólar.
- Crie um TextField que leia o valor em real e faça a conversão para dólar e apresente em um label.
- Cotação: 1 dólar = R\$5,44



Digite o valor em Reais:

Converter

Dolares:

Atividade

- Criar um conversor de real para dólar.
- Crie um TextField que leia o valor em real e faça a conversão para dólar e apresente em um label.
- Cotação: 1 dólar = R\$5,44

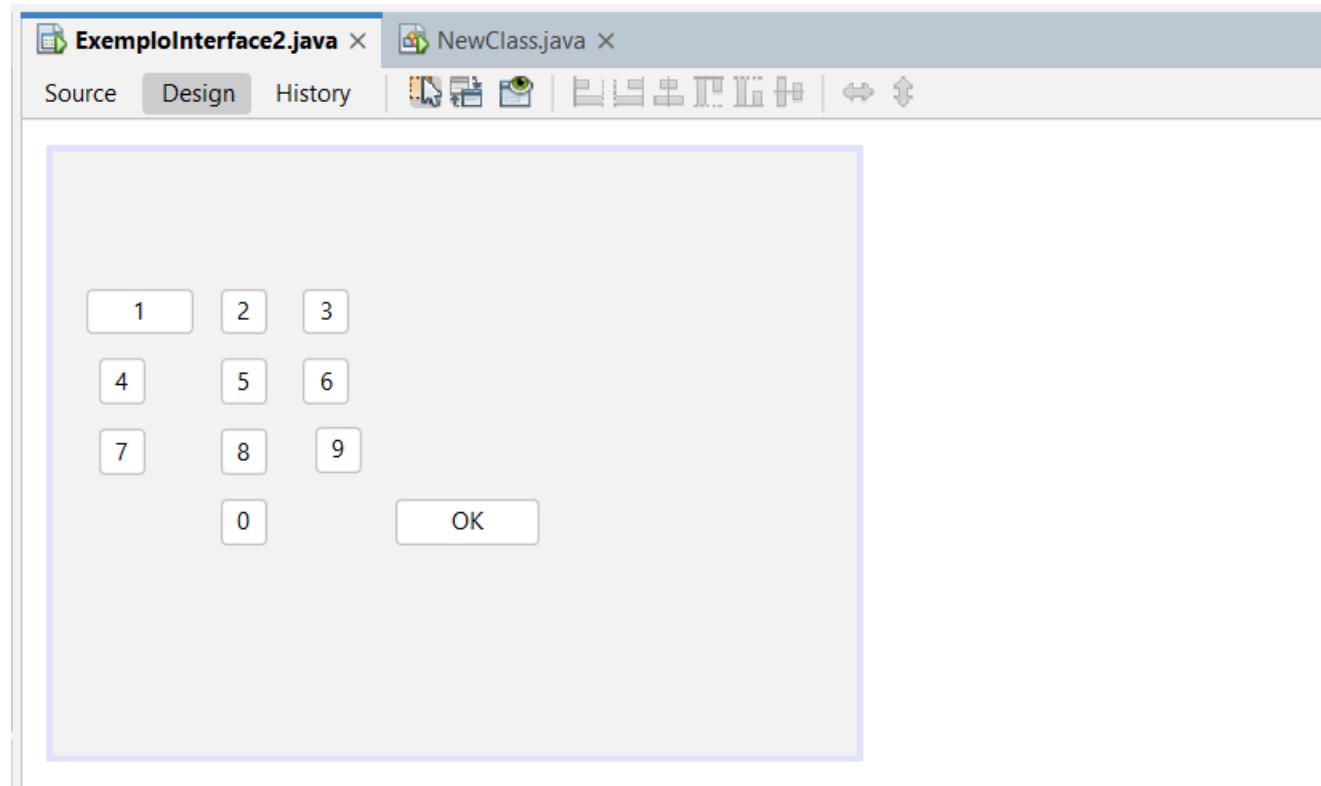
```
private void btnConverteActionPerformed(java.awt.event.ActionEvent evt) {  
    double dolares, reais;  
    reais = Double.parseDouble(txtReais.getText());  
    dolares = reais/5.44;  
    lblDolares.setText("Dolares: "+dolares);  
}
```

Java Swing

- Atividade:
 - Crie uma interface gráfica nova e adicione botões de uma calculadora...

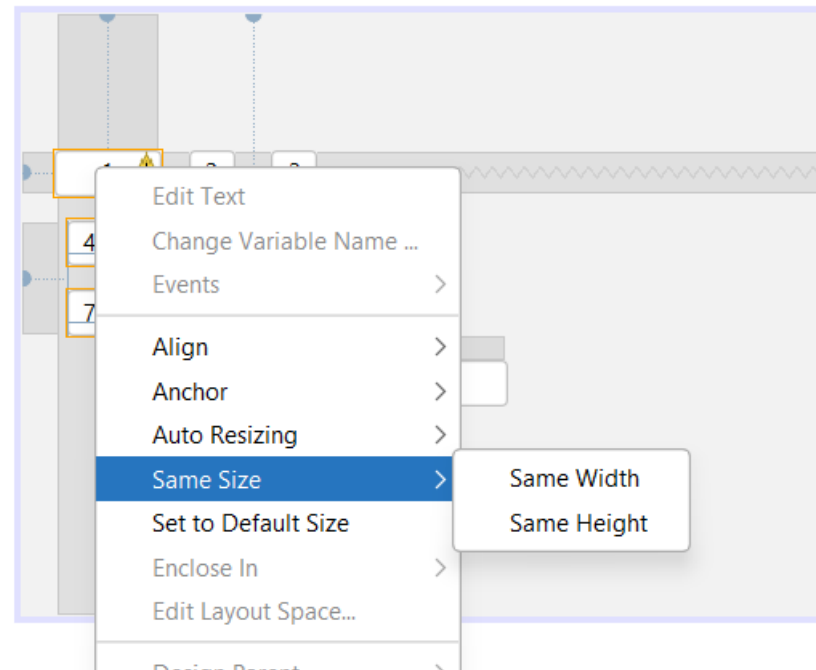
%	CE	C	$\boxtimes$
$\frac{1}{x}$	x^2	$\sqrt[3]{x}$	$\div$
7	8	9	$\times$
4	5	6	$-$
1	2	3	$+$
$\pm/-$	0	,	$=$

Java Swing



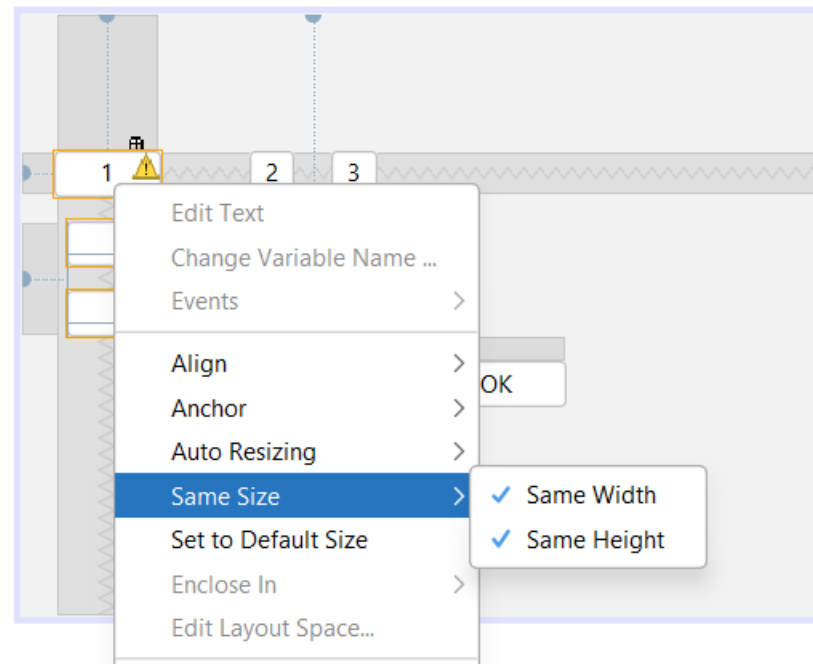
Java Swing

- Para deixar alguns componentes com o mesmo tamanho, selecione os itens apertando a tecla “Ctrl” e selecione por último a que você quer copiar o tamanho:



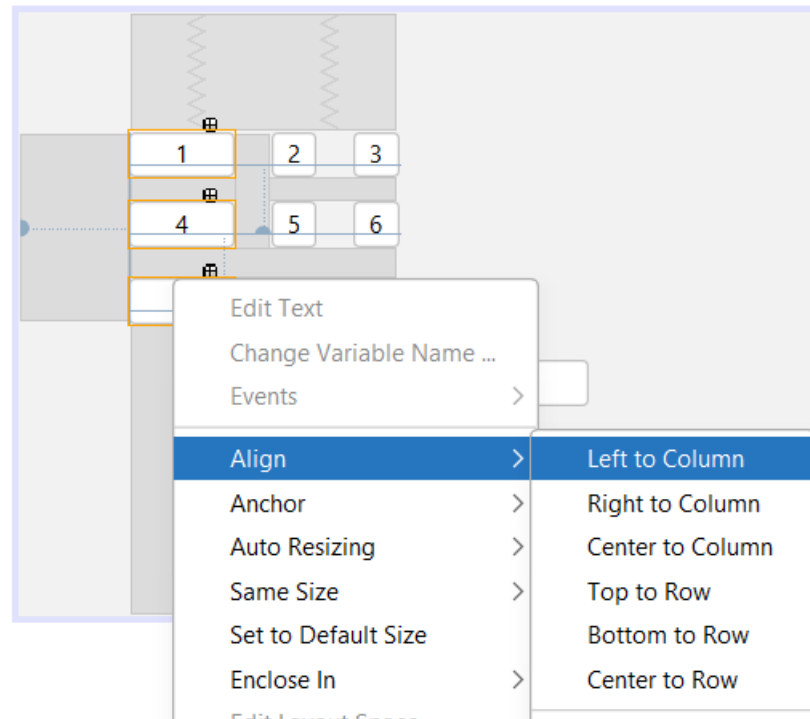
Java Swing

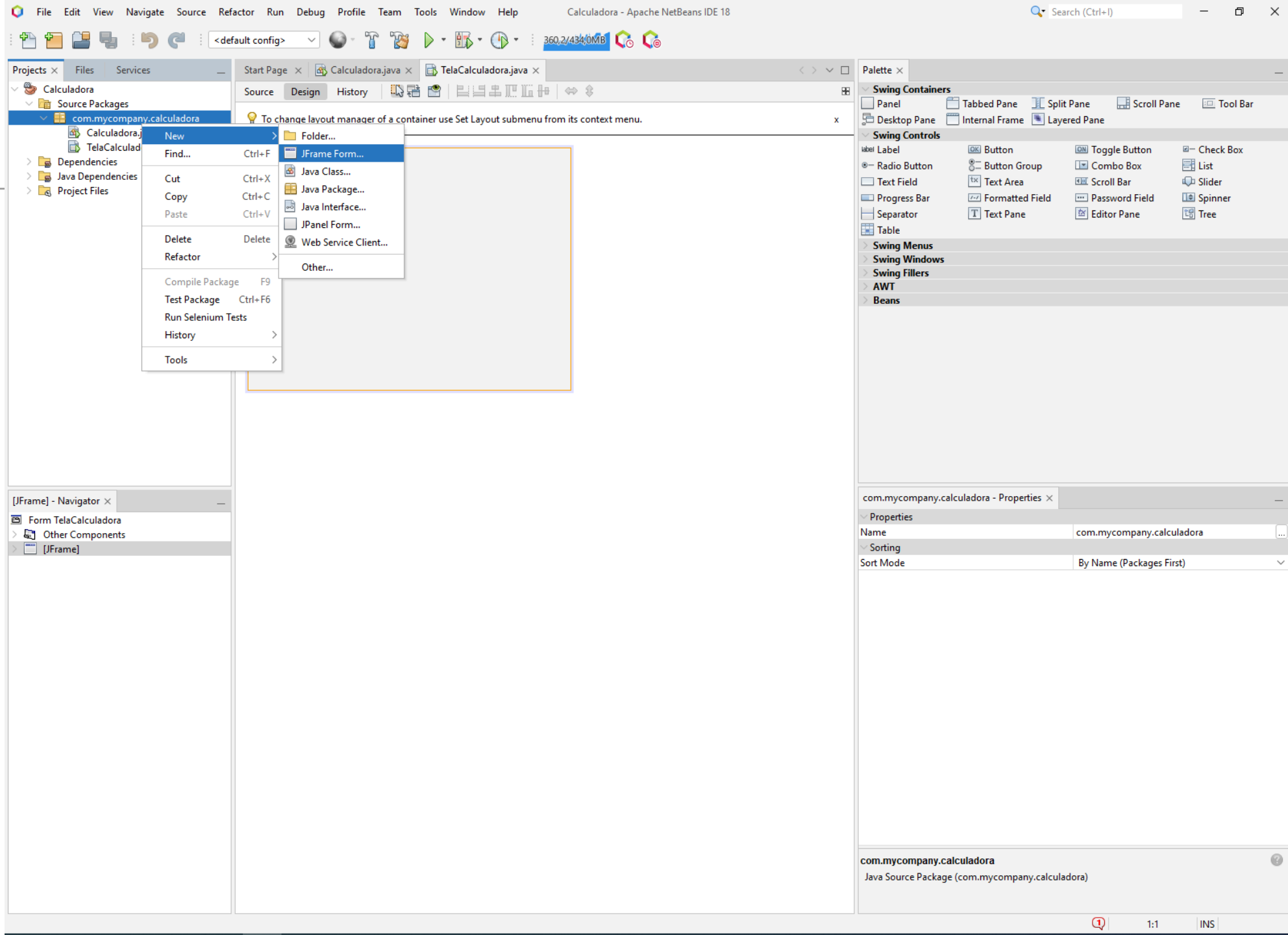
- Para deixar alguns componentes com o mesmo tamanho, selecione os itens apertando a tecla “Ctrl” e selecione por último a que você quer copiar o tamanho:



Java Swing

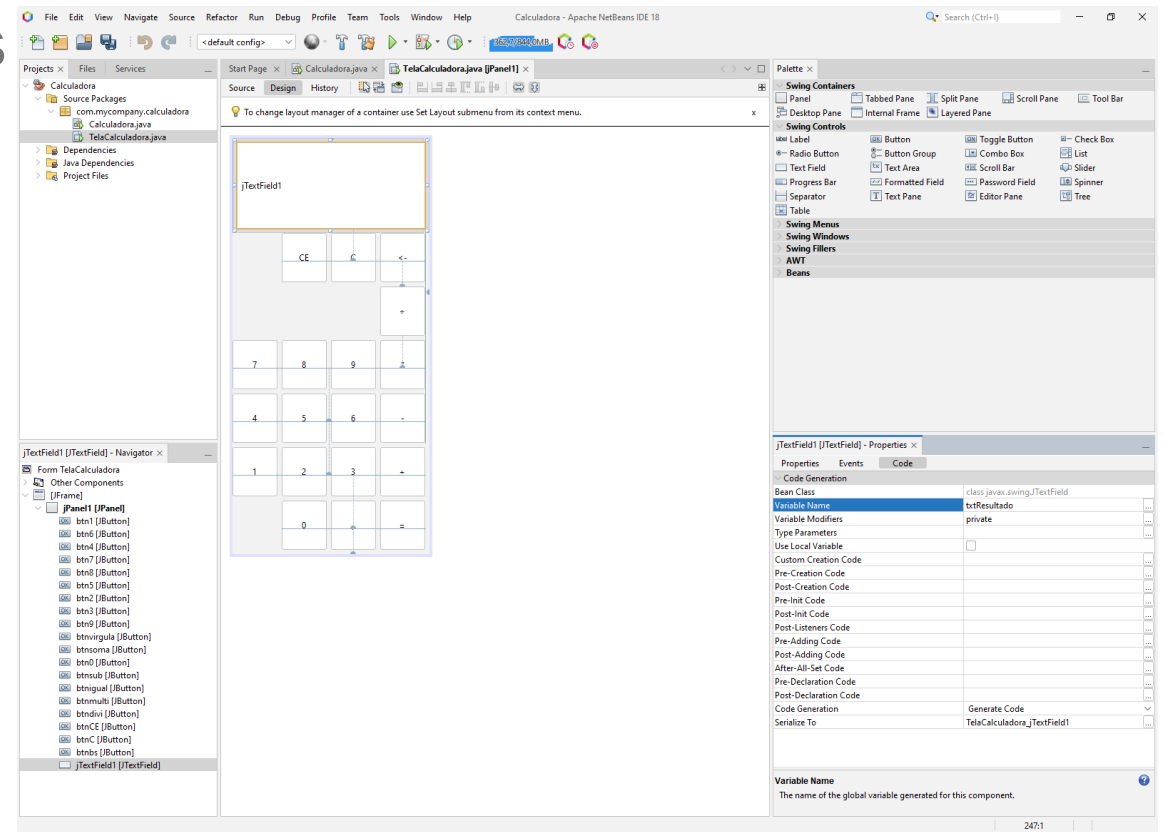
- Agora, para alinhar, faça o mesmo, selecione os componentes deixando por último o elemento o qual você quer deixar igual:



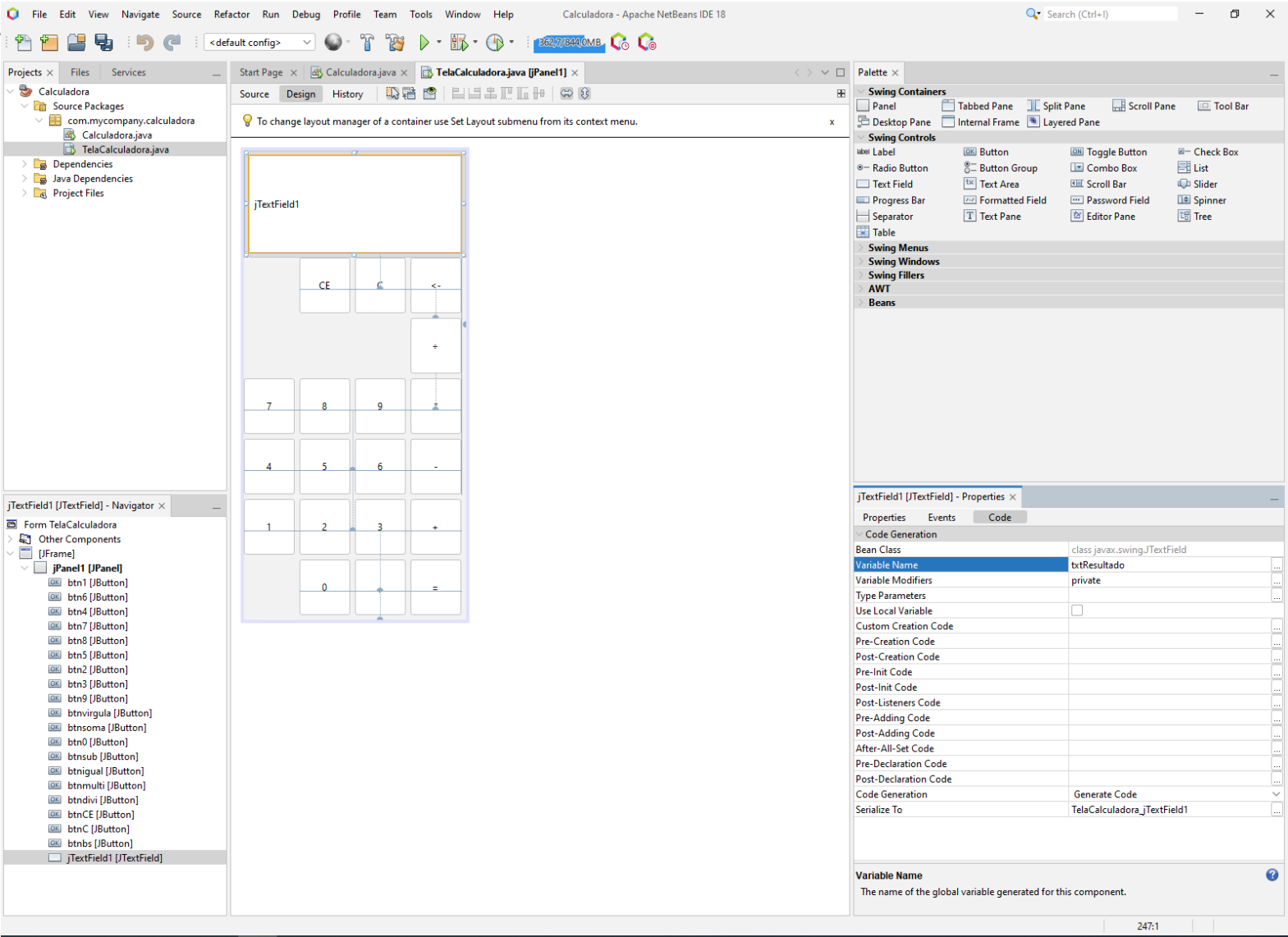


Desenvolvimento de uma calculadora

- Adicione os botões da calcular, coloque o nome conforme convenção (btnNOME)
- Coloque só as principais operações
- Adicione o Textfield



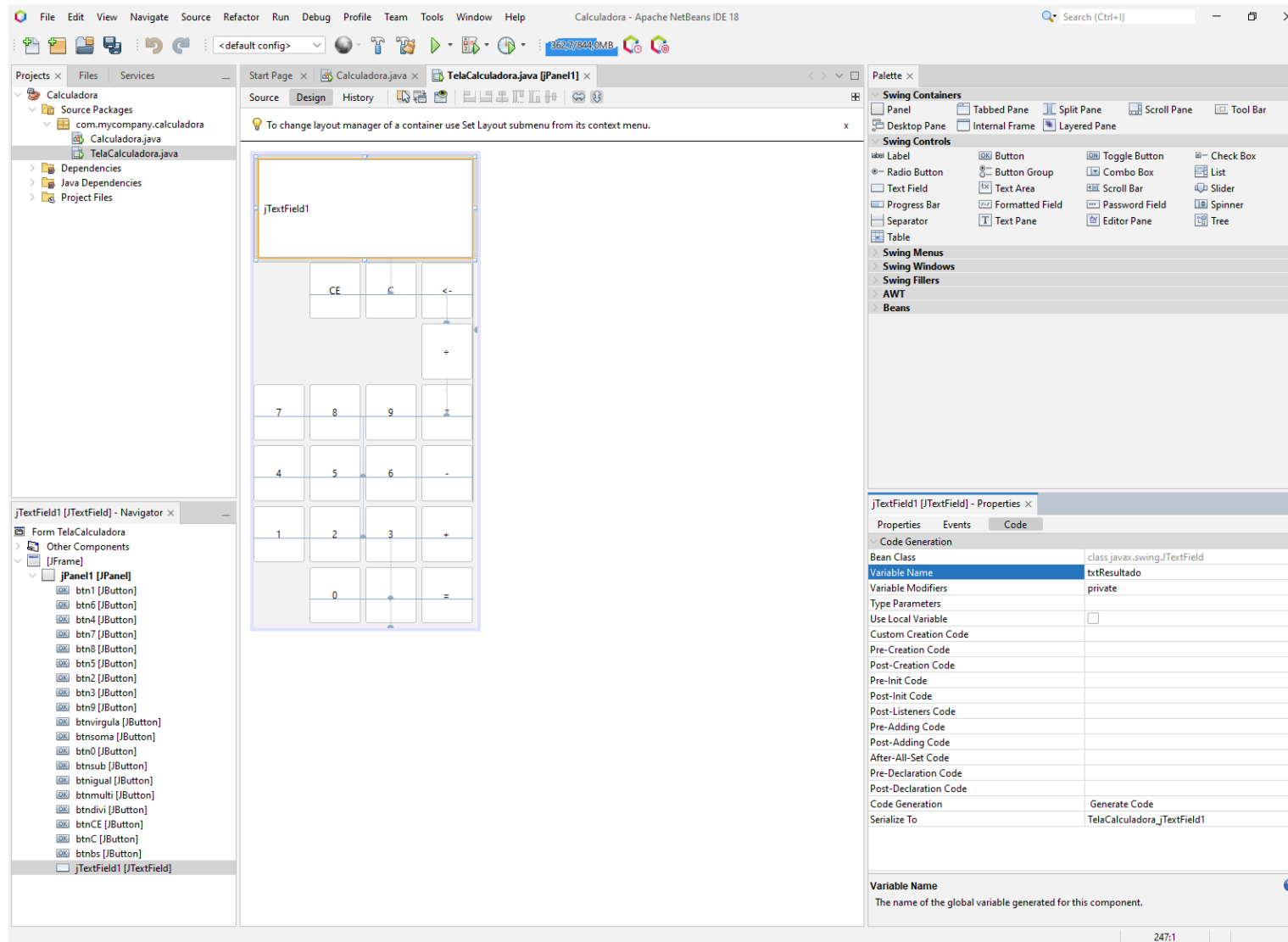
Desenvolvimento de uma calculadora



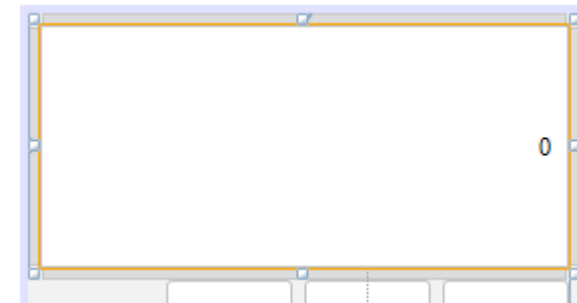
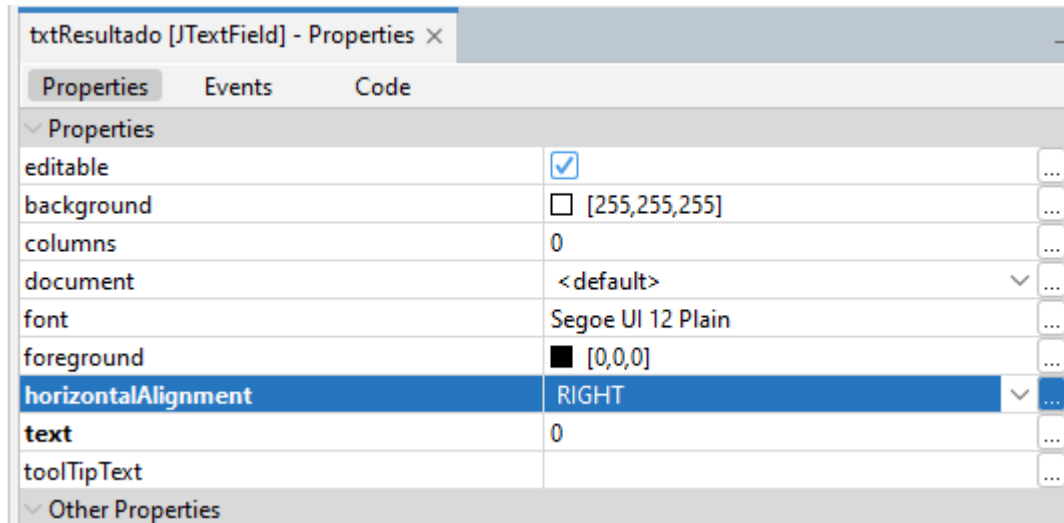
Adicione os outros botões....

- Dificuldades em manter os botões na posição?
 - Adicione um Panel e coloque os botões dentro, isso vai ajudar...

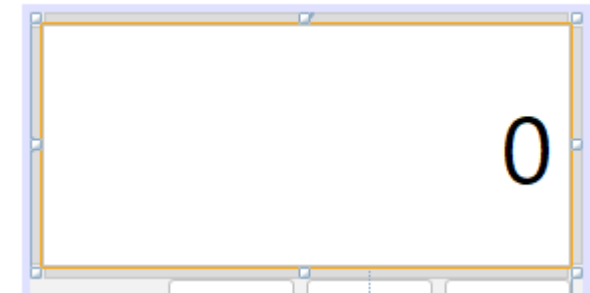
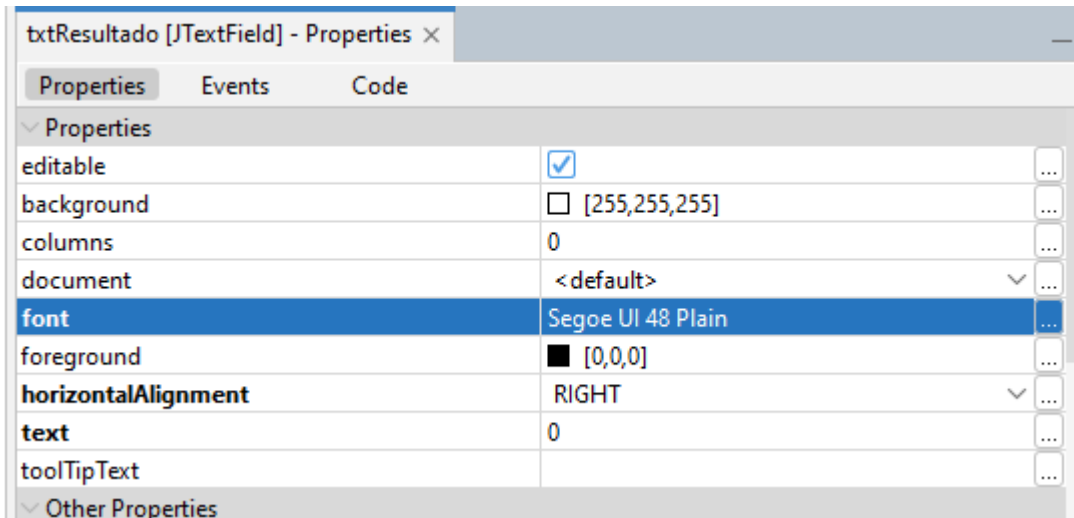
Desenvolvimento de uma calculadora



Desenvolvimento de uma calculadora



Desenvolvimento de uma calculadora



Agora, precisamos configurar as ações dos botões

- No evento de ação (do clique) do botão 0 (btn0) adicionei a chamada da função setText do TextBox txtResultado:

```
256 private void btn0ActionPerformed(java.awt.event.ActionEvent evt) {  
258     txtResultado.setText(e: "0");  
259 }  
...
```

- Podemos fazer isso nos outros botões:

```
277 private void btn1ActionPerformed(java.awt.event.ActionEvent evt) {  
278     txtResultado.setText(e: "1");  
279 }  
280 private void btn2ActionPerformed(java.awt.event.ActionEvent evt) {  
281     txtResultado.setText(e: "2");  
282 }  
283
```

Desenvolvendo uma calculadora

- Agora teste o clique no botão...
 - O que acontece quando você aperta um botão e depois o outro?

Desenvolvendo uma calculadora

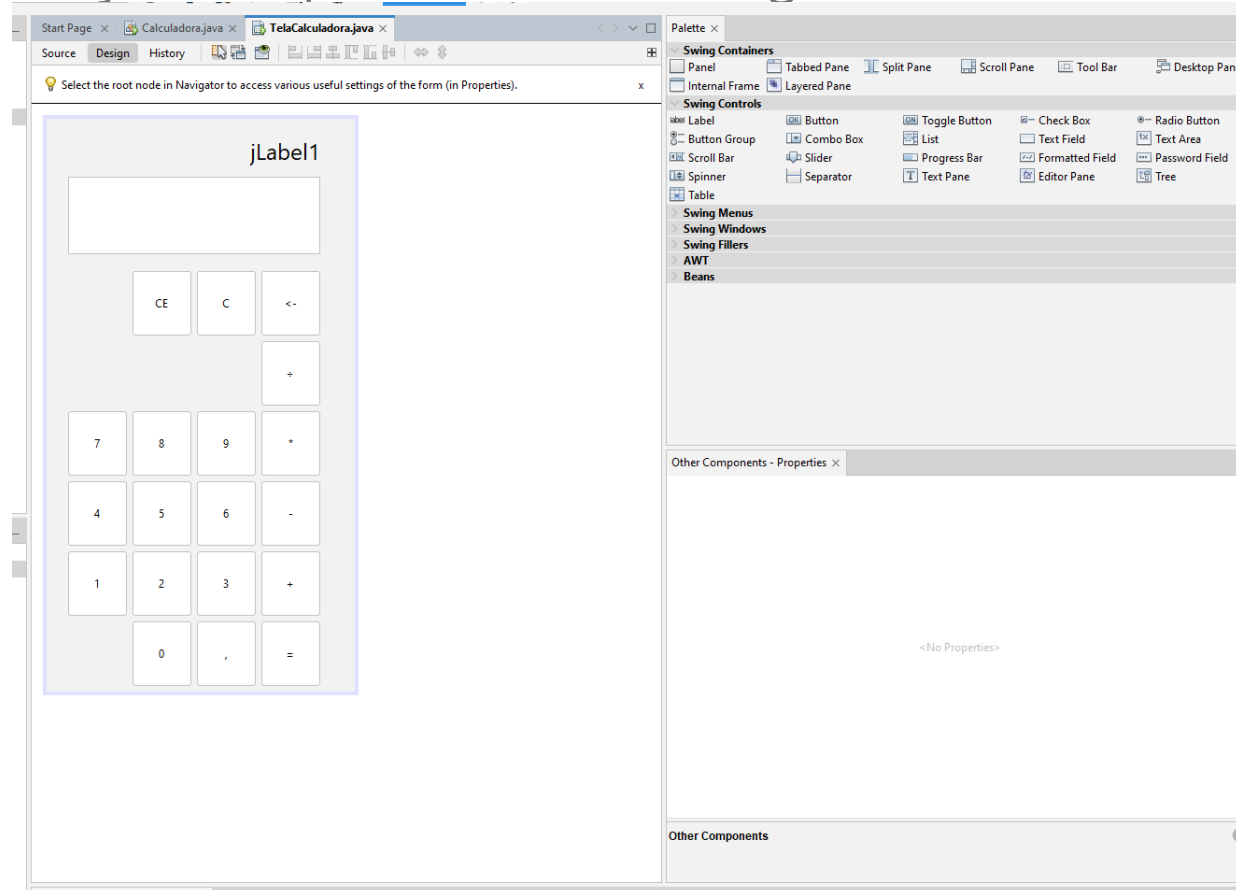
- Agora teste o clique no botão...
- O que acontece quando você aperta um botão e depois o outro?
 - Sobrescreve o valor, certo?
- Para isso, precisamos melhorar nosso código..

Desenvolvendo uma calculadora

```
273 private void btn0ActionPerformed(java.awt.event.ActionEvent evt) {  
274     txtResultado.setText(txtResultado.getText()+"0");  
275 }  
276  
277 private void btn1ActionPerformed(java.awt.event.ActionEvent evt) {  
278     txtResultado.setText(txtResultado.getText()+"1");  
279 }  
280  
281 private void btn2ActionPerformed(java.awt.event.ActionEvent evt) {  
282     txtResultado.setText(txtResultado.getText()+"2");  
283 }  
284  
285 private void btn3ActionPerformed(java.awt.event.ActionEvent evt) {  
286     txtResultado.setText(txtResultado.getText()+"3");  
287 }
```

Desenvolvendo uma calculadora

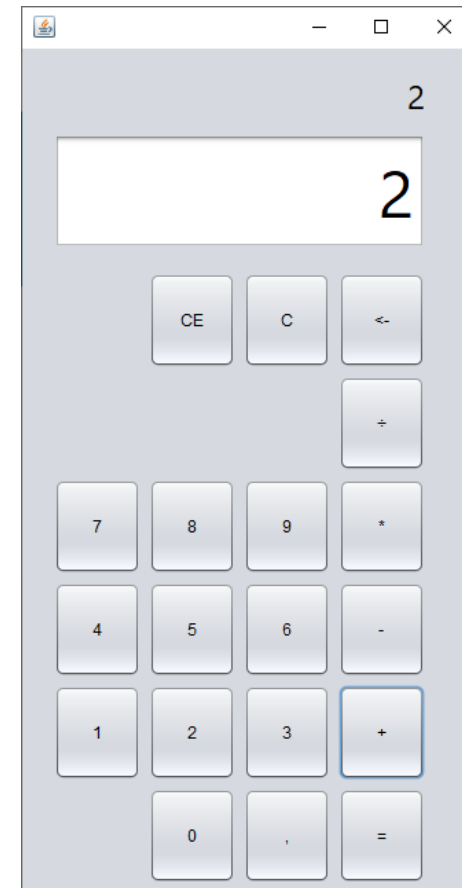
- Agora, precisamos implementar as operações, mas antes, para lembrarmos e termos a informação do primeiro valor digitado, vamos adicionar um label



Desenvolvendo uma calculadora

- Renomeei um label para lblValor1, e fui pro actionPerformed da soma (dei dois cliques no botão +)

```
331  
333 private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
334     lblValor1.setText(txtResultado.getText());  
}
```

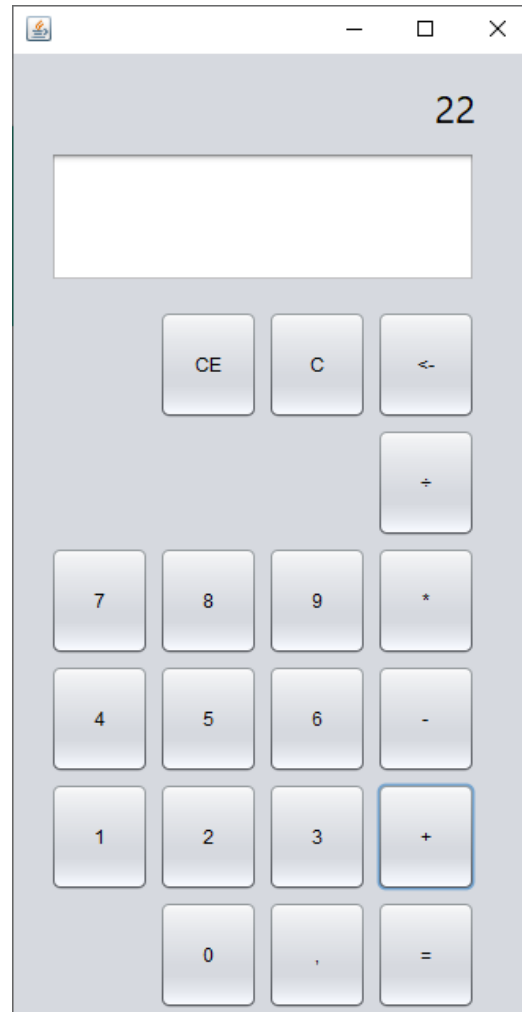


Desenvolvendo uma calculadora

- Mas ainda, precisamos limpar o texto do txtResultado

```
333 private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
335     lblValor1.setText(text: txtResultado.getText());  
336     txtResultado.setText(text: "");  
337  
338 }
```

Desenvolvendo uma calculadora



Desenvolvendo uma calculadora

- E ainda vamos precisar de algumas variáveis novas
- Declaremos como globais as variáveis valor1 e valor2 do tipo double e
- Uma String operacao para armazenar a operação que será efetuada

```
15  
16 public TelaCalculadora() {  
17     initComponents();  
18 }  
19  
20 double valor1, valor2;  
21 String operacao;  
22
```

Desenvolvendo uma calculadora

- E agora, no ActionPerformed do botão soma, vamos adicionar as atribuições das variáveis criadas:

```
335 private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
336     lblValor1.setText(text: txtResultado.getText());  
337     txtResultado.setText(t: "");  
338     valor1 = Double.parseDouble(s: lblValor1.getText());  
339     operacao = "soma";  
340  
341 }
```


Desenvolvendo uma calculadora

- E agora, precisamos manipular no botão “=” para calcular e apresentar o resultado...

```
private void btnigualActionPerformed(java.awt.event.ActionEvent evt) {  
    357     double resultado;  
    358     valor2 = Double.parseDouble(s: txtResultado.getText());  
    if(operacao=="soma") {  
    360         resultado = valor1 + valor2;  
    361         lblValor1.setText (valor1+"+"+valor2+"=");  
    362         txtResultado.setText (e: String.valueOf(d: resultado));  
    363     }  
    364 }
```

E o restante..

- É com vocês...

Desenvolvendo uma calculadora

```
private void btnigualActionPerformed(java.awt.event.ActionEvent evt) {  
    353     double resultado;  
    354     valor2 = Double.parseDouble(s: txtResultado.getText());  
    355     if(operacao=="soma"){  
    356         resultado = valor1 + valor2;  
    357         lblValor1.setText (valor1+"+"+valor2+"=");  
    358         txtResultado.setText(s: String.valueOf(d: resultado));  
    359     }  
    360     else if(operacao=="sub"){  
    361         resultado = valor1 - valor2;  
    362         lblValor1.setText (valor1+"-"+valor2+"=");  
    363         txtResultado.setText(s: String.valueOf(d: resultado));  
    364     }  
    365     else if(operacao=="multi"){  
    366         resultado = valor1 * valor2;  
    367         lblValor1.setText (valor1+"x"+valor2+"=");  
    368         txtResultado.setText(s: String.valueOf(d: resultado));  
    369     }  
    370     else if(operacao=="div"){  
    371         resultado = valor1 / valor2;  
    372         lblValor1.setText (valor1+"÷"+valor2+"=");  
    373         txtResultado.setText(s: String.valueOf(d: resultado));  
    374     }  
    375 }
```

Desenvolvendo uma calculadora

```
394  
396 private void btnsubActionPerformed(java.awt.event.ActionEvent evt) {  
397     lblValor1.setText(text: txtResultado.getText());  
398     txtResultado.setText(t: "");  
399     valor1 = Double.parseDouble(s: lblValor1.getText());  
400     operacao = "sub";  
401 }
```

```
388 private void btnmultiActionPerformed(java.awt.event.ActionEvent evt) {  
389     lblValor1.setText(text: txtResultado.getText());  
390     txtResultado.setText(t: "");  
391     valor1 = Double.parseDouble(s: lblValor1.getText());  
392     operacao = "multi"; // TODO add your handling code here:  
393 }  
394
```

```
306 private void btndiviActionPerformed(java.awt.event.ActionEvent evt) {  
307     lblValor1.setText(text: txtResultado.getText());  
308     txtResultado.setText(t: "");  
309     valor1 = Double.parseDouble(s: lblValor1.getText());  
310     operacao = "div";  
311 }
```

Desenvolvendo uma calculadora

- E o C e o CE?
- E as outras operações?
- Agora sim, é com vocês, façam conforme a curiosidade e a vontade de vocês..

Exercícios

- 1) Finalizar as outras operações da calculadora (no mínimo as 4 operações, o C , CE e o backspace).
- 2) Criar uma interface para cálculo do IMC de uma pessoa. Receba os valores em um textbox e calcule e apresente o resultado em um label.

```
411 private void btnigualActionPerformed(java.awt.event.ActionEvent evt) {  
412     double resultado;  
413     String valorFormat;  
414     valor2 = Double.parseDouble(txtResultado.getText());  
415     if (operacao.equals("soma")) {  
416         resultado = valor1 + valor2;  
417         lblValor1.setText(valor1 + "+" + valor2 + "=");  
418         valorFormat = String.format("%.2f", resultado);  
419         txtResultado.setText(valorFormat);  
420     }  
421     if (operacao.equals("sub")) {  
422         resultado = valor1 - valor2;  
423         lblValor1.setText(valor1 + "-" + valor2 + "=");  
424         valorFormat = String.format("%.2f", resultado);  
425         txtResultado.setText(valorFormat);  
426     }  
427     if (operacao.equals("div")) {  
428         resultado = valor1 / valor2;  
429         lblValor1.setText(valor1 + "/" + valor2 + "=");  
430         valorFormat = String.format("%.2f", resultado);  
431         txtResultado.setText(valorFormat);  
432     }  
433     if (operacao.equals("multi")) {  
434         resultado = valor1 * valor2;  
435         lblValor1.setText(valor1 + "x" + valor2 + "=");  
436         valorFormat = String.format("%.2f", resultado);  
437         txtResultado.setText(valorFormat);  
438     }  
439 }
```

```
404 private void btnsomaActionPerformed(java.awt.event.ActionEvent evt) {  
405     valor1 = Double.parseDouble(txtResultado.getText());  
406     lblValor1.setText(txtResultado.getText() + "+");  
407     txtResultado.setText("");  
408     operacao = "soma";  
409 }
```

```
441 private void btrndivActionPerformed(java.awt.event.ActionEvent evt) {  
442     valor1 = Double.parseDouble(txtResultado.getText());  
443     lblValor1.setText(txtResultado.getText() + "/");  
444     txtResultado.setText("");  
445     operacao = "div";  
446 }
```

```
448 private void btnsubActionPerformed(java.awt.event.ActionEvent evt) {  
449     valor1 = Double.parseDouble(txtResultado.getText());  
450     lblValor1.setText(txtResultado.getText() + "-");  
451     txtResultado.setText("");  
452     operacao = "sub";  
453 }
```

```
455 private void btnmultiActionPerformed(java.awt.event.ActionEvent evt) {  
456     valor1 = Double.parseDouble(txtResultado.getText());  
457     lblValor1.setText(txtResultado.getText() + "x");  
458     txtResultado.setText("");  
459     operacao = "multi";  
460 }
```



```
462 private void btnCEActionPerformed(java.awt.event.ActionEvent evt) {  
463     valor2 = 0;  
464     txtResultado.setText("");  
465 }  
466  
467 private void btnCAActionPerformed(java.awt.event.ActionEvent evt) {  
468     valor1 = 0;  
469     lblValor1.setText("");  
470     valor2 = 0;  
471     txtResultado.setText("");  
472 }  
473  
474 private void btnBSActionPerformed(java.awt.event.ActionEvent evt) {  
475     String valor = txtResultado.getText();  
476     if(txtResultado.getText().length()>0)  
477         txtResultado.setText(valor.substring(0, valor.length() - 1));  
478 }
```